

Culture & Outils DevOps

CI/CD · Docker · Kubernetes

Formateur : Haithem Mihoubi

Module : 2 / 3

Durée : 4 jours (28 heures)

Niveau : Mixte (débutant → intermédiaire)

Public : développeurs, ops, futurs ingénieurs DevOps

Pré-requis : bases de la ligne de commande Linux et de Git

Sommaire

Avant de commencer	4
Jour 1 — Culture DevOps & CI/CD	5
1.1 Origine et philosophie DevOps	5
1.2 Le cycle de vie DevOps	6
1.3 Git & Git Flow (rappels utiles)	6
1.4 Anatomie d'un pipeline CI	7
1.5 Atelier — Pipeline CI	7
Quiz — Jour 1	9
Jour 2 — Docker : conteneurisation	10
2.1 Conteneurs vs machines virtuelles	10
2.2 Concepts Docker	10
2.3 Le Dockerfile	11
2.4 Docker Compose	12
2.5 Registres d'images	13
2.6 Atelier — Conteneuriser une application	13
Quiz — Jour 2	13
Jour 3 — Introduction à Kubernetes	15
3.1 Pourquoi un orchestrateur ?	15
3.2 Architecture et objets	15
3.3 Configuration : ConfigMaps, Secrets, Namespaces	17
3.4 RBAC & sécurité	18
3.5 Atelier — Déployer sur Minikube / K3s	18
Quiz — Jour 3	19
Jour 4 — Automation & DevOps avancé	20
4.1 Pipeline CI/CD de bout en bout	20
4.2 Helm Charts	21
4.3 Observabilité : Prometheus & Grafana	21
4.4 GitOps avec Argo CD	22
4.5 Atelier final — Chaîne complète CI/CD → Docker → Kubernetes	23
Partie pratique approfondie — Linux, Docker, Kubernetes, GitHub & CI/CD	24
Atelier Linux — le système, le terminal et le shell	24
Atelier API REST & documentation (OpenAPI / Swagger)	31
Atelier Tests automatisés — le filet qui rend la CI utile	34
Atelier Migrations de base de données (Flyway)	37
Atelier Docker pratique — de l'image au conteneur	40
Atelier Kubernetes pratique — déployer pour de vrai	45

Atelier GitHub — la plateforme de collaboration	49
Atelier GitHub Actions — CI/CD de bout en bout	51
Atelier DevSecOps — sécuriser la chaîne de livraison	55
Atelier Terraform — l'infrastructure en tant que code (IaC)	58
Évaluation finale du Module 2	61

Avant de commencer

Ce module est **très pratique** : vous tapez réellement chaque commande au fur et à mesure. Le fil rouge est une petite API web (« **QuickBite API** », Spring Boot / Java) que l'on construit, conteneurise, déploie, puis livre via un pipeline complet.

⚠ Préparer votre poste

Installez avant de démarrer : **Git**, **Docker** (Docker Desktop ou Docker Engine + Compose), un compte **GitHub** (ou GitLab), **kubectI**, et **Minikube** (ou **K3s/kind**). Si une machine pose problème, une solution « cloud » dépanne très bien : GitHub Codespaces, Killercoda ou Play with Docker.

Vérification d'environnement (à exécuter au démarrage)

```
git --version
docker --version && docker compose version
kubectI version --client
minikube version    # ou: k3s --version
```

Jour 1 — Culture DevOps & CI/CD

🎯 Objectifs du Jour 1

- Expliquer ce qu'est (et n'est pas) le DevOps et le modèle CAMS.
- Décrire le cycle de vie DevOps et la place de l'automatisation.
- Maîtriser un flux Git collaboratif (branches, PR/MR).
- Comprendre l'anatomie d'un pipeline d'intégration continue.
- Construire un premier pipeline CI qui teste automatiquement le code.

1.1 Origine et philosophie DevOps

Le **DevOps** est né du divorce historique entre les **développeurs** (qui veulent livrer du changement) et les **opérations** (qui veulent de la stabilité). Ce « mur de la confusion » provoquait des déploiements rares, risqués et conflictuels. Le mouvement DevOps (popularisé vers 2009, *DevOpsDays*) propose de **réunir Dev et Ops** par la culture et l'outillage.

📖 Définition de travail

DevOps n'est **ni un métier, ni un outil** : c'est une **culture** et un ensemble de pratiques visant à raccourcir le cycle entre une idée et sa mise en production, de façon fiable et répétable.

Le modèle CAMS

Pilier	Idée	Exemple concret
C — Culture	Collaboration, responsabilité partagée	Dev et Ops « on call » ensemble
A — Automation	Automatiser ce qui est répétitif	Build, tests, déploiement
M — Measurement	Mesurer pour décider	Lead time, taux d'échec, MTTR
S — Sharing	Partager savoir et feedback	Post-mortems sans blâme

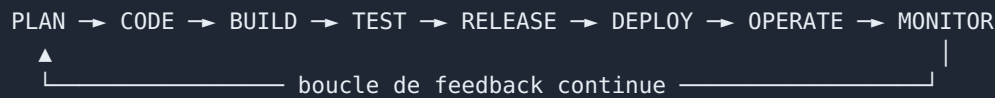
Les métriques DORA (référence du secteur)

1. **Deployment Frequency** — à quelle fréquence on déploie.
2. **Lead Time for Changes** — du commit à la production.
3. **Change Failure Rate** — % de déploiements provoquant un incident.
4. **MTTR** (Mean Time To Restore) — temps de remise en service.

💡 **CI ≠ CD ≠ CD**

CI = Intégration Continue (on intègre/teste à chaque commit). **CD** = Livraison Continue (l'artefact est toujours *déployable*, déploiement manuel). **CD** = Déploiement Continu (déploiement *automatique* en prod). On gravit ces marches dans cet ordre.

1.2 Le cycle de vie DevOps



- **Plan / Code** : backlog, branches, revue.
- **Build / Test** : compilation, tests automatisés, analyse de qualité/sécurité.
- **Release / Deploy** : artefact versionné, déploiement (manuel ou auto).
- **Operate / Monitor** : exploitation, supervision, alertes → retour au Plan.

1.3 Git & Git Flow (rappels utiles)

Commandes essentielles

```

git init                # initialiser un dépôt
git clone <url>          # cloner
git switch -c feature/paiement # créer + basculer sur une branche
git add . && git commit -m "feat: ajout du paiement"
git push -u origin feature/paiement
# puis ouvrir une Pull Request / Merge Request
  
```

Convention de messages de commit (Conventional Commits)

```

feat:    nouvelle fonctionnalité
fix:     correction de bug
docs:    documentation
refactor: refonte sans changement fonctionnel
test:    ajout/modif de tests
chore:   tâches techniques (build, deps)
  
```

Stratégies de branches

Stratégie	Principe	Pour qui
Git Flow	Branches <code>main</code> , <code>develop</code> , <code>feature/*</code> , <code>release/*</code> , <code>hotfix/*</code>	Releases planifiées
GitHub Flow	<code>main</code> + branches courtes + PR + déploiement continu	Web/SaaS, livraison fréquente
Trunk-Based	Commits fréquents sur <code>main</code> , feature flags	Équipes matures, CI forte

⚠ Branches longues = douleur d'intégration

Plus une branche vit longtemps, plus la fusion est risquée. La CI encourage des branches **courtes** et des intégrations **fréquentes**.

1.4 Anatomie d'un pipeline CI

Un pipeline est une suite d'**étapes (stages)** déclenchées par un événement (push, PR). Étapes typiques :

```
push → [checkout] → [install deps] → [lint] → [build] → [tests] → [artefact] → (✓/✗)
```

Principes : **rapide** (feedback en minutes), **reproductible** (mêmes versions partout), **fail fast** (échouer tôt), **vert obligatoire** avant fusion.

1.5 Atelier — Pipeline CI

🔧 Atelier 1 — Durée 2 h — En binômes

Mission : créer un dépôt avec une petite application, écrire un test, et mettre en place un pipeline CI qui s'exécute à chaque push et bloque la fusion si les tests échouent.

Variante A — GitHub Actions (Node.js)

Fichier `.github/workflows/ci.yml` :

```
name: CI
on:
  push:
    branches: [ main ]
  pull_request:
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
      - run: npm ci
      - run: npm run lint --if-present
      - run: npm test
```

Test minimal (`sum.test.js` avec Jest) :

```
const sum = (a, b) => a + b;

test('additionne deux nombres', () => {
  expect(sum(2, 3)).toBe(5);
});
```

Variante B — GitLab CI

Fichier `.gitlab-ci.yml` :

```
stages: [build, test]

build:
  stage: build
  image: node:20
  script:
    - npm ci
    - npm run build --if-present

test:
  stage: test
  image: node:20
  script:
    - npm ci
    - npm test
```

Étapes guidées : 1. (20 min) Créer le dépôt, ajouter l'app et le test ; vérifier `npm test` en local. 2. (30 min) Ajouter le fichier de workflow et pousser ; observer l'exécution. 3. (20 min) **Casser** volontairement le test → constater le pipeline rouge. 4. (20 min) Protéger la branche `main` (fusion interdite si CI rouge). 5. (30 min) Restitution + lecture des logs.

 Le moment clé de l'atelier

L'étape la plus instructive est celle où vous voyez une fusion **bloquée** par un test rouge : c'est ce qui fait comprendre, concrètement, la valeur de la CI. Observez aussi le **cache des dépendances** (le second run est bien plus rapide) et l'exécution du pipeline **sur une Pull Request**.

Quiz — Jour 1

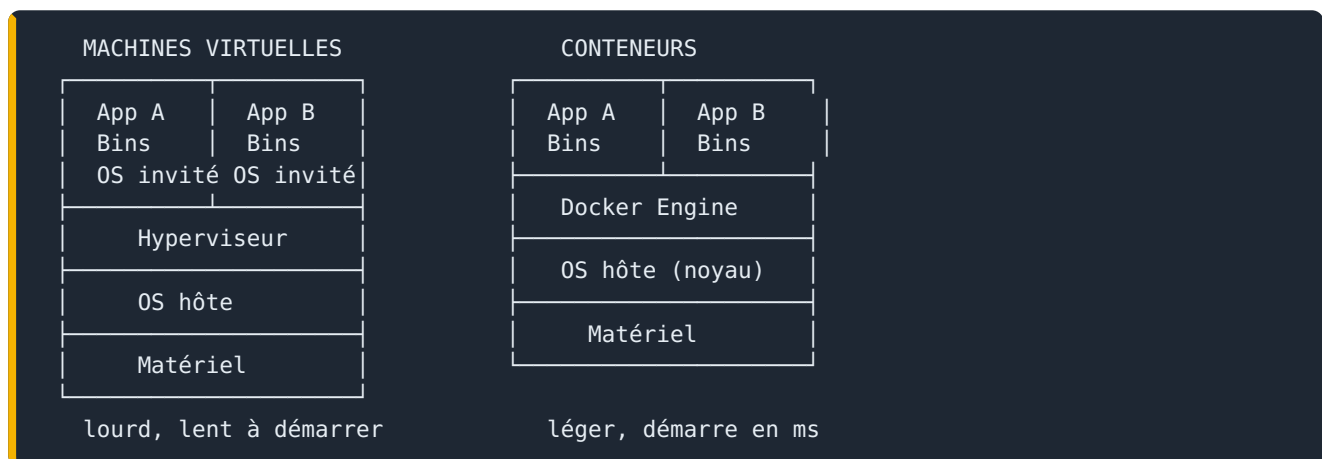
1. Que veut dire CAMS ?
 2. Donnez deux des quatre métriques DORA.
 3. Différence entre Livraison Continue et Déploiement Continu ?
 4. Pourquoi préférer des branches courtes ?
 5. Que fait `git switch -c maBranche` ?
-

Jour 2 — Docker : conteneurisation

🎯 Objectifs du Jour 2

- Expliquer la différence entre conteneur et machine virtuelle.
- Manipuler images, conteneurs, volumes et réseaux Docker.
- Écrire un Dockerfile propre et optimisé (multi-stage).
- Orchestrer plusieurs services en local avec Docker Compose.
- Publier une image sur un registre.

2.1 Conteneurs vs machines virtuelles



Un conteneur **partage le noyau** de l'hôte et n'embarque que l'application + ses dépendances : il est **léger, rapide, portable** (« ça marche chez moi » → « ça marche partout »).

2.2 Concepts Docker

Objet	Définition	Analogie
Image	Modèle en lecture seule (couches)	Le « moule » / une classe
Conteneur	Instance en exécution d'une image	L'objet / le gâteau
Volume	Stockage persistant hors du conteneur	Disque externe
Réseau	Réseau virtuel reliant des conteneurs	Le LAN privé
Registry	Dépôt d'images (Docker Hub, GHCR)	App store d'images

Commandes de base

```
docker run -d -p 8080:80 --name web nginx      # lancer un conteneur en arrière-plan
docker ps                                     # conteneurs actifs
docker logs -f web                            # suivre les logs
docker exec -it web sh                        # ouvrir un shell dedans
docker stop web && docker rm web               # arrêter et supprimer
docker images                                 # lister les images
docker volume create data                     # créer un volume
docker network create app-net                 # créer un réseau
```

2.3 Le Dockerfile

Exemple commenté (application Node.js), **multi-stage** pour une image finale légère :

```
# ----- Étape 1 : build -----
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci                                # installe TOUTES les deps (dont dev)
COPY . .
RUN npm run build                          # compile (ex. TypeScript -> dist/)

# ----- Étape 2 : runtime -----
FROM node:20-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
RUN npm ci --omit=dev                    # uniquement les deps de prod
COPY --from=build /app/dist ./dist
EXPOSE 3000
USER node                                # ne pas tourner en root
CMD ["node", "dist/server.js"]
```

Bonnes pratiques Dockerfile

- **Image de base minimale** (`-alpine` , `-slim` , ou `distroless`).
- **Ordonner du moins au plus changeant** pour profiter du cache de couches (copier `package.json` avant le code).
- **Multi-stage** pour ne pas embarquer les outils de build.
- `.dockerignore` (exclure `node_modules` , `.git` ...).
- **Ne pas tourner en root**, ne pas stocker de secrets dans l'image.
- **Tag explicite** des versions (éviter `latest` en prod).

```
# .dockerignore
node_modules
.git
*.log
dist
```

! Jamais de secrets dans une image

Les couches d'image sont consultables. N'y copiez **jamais** mots de passe, clés ou tokens. Utilisez des variables d'environnement à l'exécution, ou des secrets gérés (Docker/K8s secrets, coffre-fort).

2.4 Docker Compose

Compose décrit **plusieurs services** dans un fichier et les démarre ensemble. Idéal pour le développement local (app + base de données).

```
services:
  api:
    build: .
    ports:
      - "3000:3000"
    environment:
      - DATABASE_URL=postgres://app:secret@db:5432/quickbite
    depends_on:
      - db
    networks: [app-net]

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: quickbite
    volumes:
      - db-data:/var/lib/postgresql/data
    networks: [app-net]

volumes:
  db-data:
networks:
  app-net:
```

```
docker compose up -d --build    # construire et démarrer
docker compose ps               # état des services
docker compose logs -f api      # logs d'un service
docker compose down -v          # tout arrêter et supprimer les volumes
```

2.5 Registres d'images

```
# Docker Hub
docker login
docker build -t monuser/quickbite-api:1.0.0 .
docker push monuser/quickbite-api:1.0.0

# GitHub Container Registry (GHCR)
echo $TOKEN | docker login ghcr.io -u monuser --password-stdin
docker tag quickbite-api ghcr.io/monuser/quickbite-api:1.0.0
docker push ghcr.io/monuser/quickbite-api:1.0.0
```

💡 Versionner par tag sémantique

Poussez à la fois un tag précis (`1.0.0`) et un tag mouvant (`1` ou `stable`). En prod, déployez le tag **précis** pour la traçabilité.

2.6 Atelier — Conteneuriser une application

🔧 Atelier 2 — Durée 2 h 30 — En binômes

Mission : conteneuriser la QuickBite API et sa base de données, la lancer via Compose, puis publier l'image sur un registre.

Étapes : 1. (30 min) Écrire le `Dockerfile` (multi-stage) + `.dockerignore`. `docker build` puis `docker run`, tester l'endpoint `GET /health`. 2. (40 min) Ajouter une base PostgreSQL via `docker-compose.yml` ; relier l'API au service `db`. 3. (30 min) Ajouter un **volume** pour persister les données ; vérifier la persistance après `down / up`. 4. (30 min) Tagger et **pousser** l'image sur Docker Hub ou GHCR. 5. (20 min) Restitution : taille de l'image, gain du multi-stage, pièges rencontrés.

Critères de réussite (DoD de l'atelier) :

- `docker compose up` démarre API + DB sans erreur.
- L'API répond et lit/écrit en base.
- L'image finale est < 200 Mo (grâce au multi-stage).
- L'image est visible dans le registre.

⚠️ Pièges fréquents à éviter

Trois erreurs classiques : l'**API démarre avant la base** et plante (réglez-le avec `depends_on` + un `healthcheck`, ou une logique de retry) ; la **confusion entre port interne et port publié** (`-p 8080:8080`) ; l'**oubli du** `.dockerignore` qui gonfle l'image inutilement. Comparez la taille de votre image **avec et sans multi-stage** : la différence est spectaculaire.

Quiz — Jour 2

1. Conteneur vs VM : quelle est la différence clé ?

2. À quoi sert un volume ?
 3. Pourquoi un build multi-stage ?
 4. Pourquoi copier `package.json` avant le code source ?
 5. Que fait `docker compose down -v` ?
-

Jour 3 — Introduction à Kubernetes

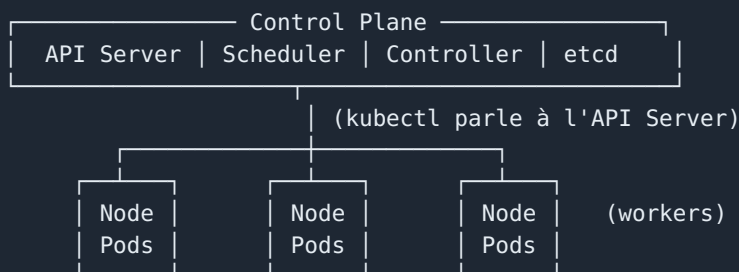
🎯 Objectifs du Jour 3

- Expliquer le rôle d'un orchestrateur de conteneurs.
- Décrire l'architecture de Kubernetes et ses objets de base.
- Déployer une application, l'exposer et la mettre à l'échelle.
- Gérer la configuration (ConfigMap/Secret) et l'isolation (Namespace).
- Comprendre les bases du RBAC et de la sécurité.

3.1 Pourquoi un orchestrateur ?

Docker lance des conteneurs **sur une machine**. En production, on veut : plusieurs machines, du **redémarrage automatique**, de la **mise à l'échelle**, des **mise à jour sans coupure**, de la **répartition de charge**, de l'**auto-réparation**. C'est le rôle de **Kubernetes (K8s)**.

3.2 Architecture et objets



Objet	Rôle
Pod	Plus petite unité déployable : un (ou +) conteneurs partageant réseau/stockage
ReplicaSet	Maintient <i>N</i> copies d'un Pod
Deployment	Gère les ReplicaSet : mises à jour progressives, rollback
Service	Adresse réseau stable + load balancing vers des Pods
Ingress	Routage HTTP(S) externe vers des Services (noms de domaine, chemins)

Un Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: quickbite-api
spec:
  replicas: 3
  selector:
    matchLabels: { app: quickbite-api }
  template:
    metadata:
      labels: { app: quickbite-api }
    spec:
      containers:
        - name: api
          image: ghcr.io/monuser/quickbite-api:1.0.0
          ports:
            - containerPort: 3000
          readinessProbe:
            httpGet: { path: /health, port: 3000 }
            initialDelaySeconds: 5
          resources:
            requests: { cpu: "100m", memory: "128Mi" }
            limits: { cpu: "500m", memory: "256Mi" }
```

Un Service

```
apiVersion: v1
kind: Service
metadata:
  name: quickbite-api
spec:
  selector: { app: quickbite-api }
  ports:
    - port: 80
      targetPort: 3000
  type: ClusterIP      # interne ; NodePort/LoadBalancer pour exposer
```


Un Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: quickbite-ingress
spec:
  rules:
    - host: quickbite.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: quickbite-api
                port: { number: 80 }
```

Commandes kubectl essentielles

```
kubectl apply -f deployment.yaml      # créer/mettre à jour depuis un manifeste
kubectl get pods,svc,deploy           # lister les objets
kubectl describe pod <nom>            # détails + événements (debug)
kubectl logs -f <pod>                 # logs
kubectl scale deploy/quickbite-api --replicas=5
kubectl rollout status deploy/quickbite-api
kubectl rollout undo deploy/quickbite-api # rollback
kubectl port-forward svc/quickbite-api 8080:80 # accès local rapide
```

■ Déclaratif, pas impératif

On décrit **l'état souhaité** dans des YAML, et Kubernetes **converge** en continu vers cet état (auto-réparation). C'est la base de la philosophie GitOps vue au Jour 4.

3.3 Configuration : ConfigMaps, Secrets, Namespaces

```
apiVersion: v1
kind: ConfigMap
metadata: { name: quickbite-config }
data:
  APP_MODE: "production"
  PAGE_SIZE: "20"
  ---
apiVersion: v1
kind: Secret
metadata: { name: quickbite-secret }
type: Opaque
stringData:
  DATABASE_PASSWORD: "secret-a-changer"
```

Injection dans un conteneur :

```
envFrom:
- configMapRef: { name: quickbite-config }
- secretRef:    { name: quickbite-secret }
```

⚠ Les Secrets K8s sont encodés, pas chiffrés

Par défaut un Secret est seulement encodé en base64. Activez le chiffrement *at rest* de etcd et/ou utilisez un gestionnaire externe (Sealed Secrets, External Secrets, Vault). Ne commitez jamais un Secret en clair.

Namespaces : cloisonnent les ressources (ex. `dev`, `staging`, `prod`) et servent de périmètre aux quotas et au RBAC.

```
kubectl create namespace dev
kubectl apply -f deployment.yaml -n dev
```

3.4 RBAC & sécurité

Le **RBAC** (Role-Based Access Control) définit *qui* peut faire *quoi* sur *quelles* ressources.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { namespace: dev, name: lecteur-pods }
rules:
- apiGroups: ["" ]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { namespace: dev, name: bind-lecteur }
subjects:
- kind: User
  name: alice
roleRef:
  kind: Role
  name: lecteur-pods
  apiGroup: rbac.authorization.k8s.io
```

Bonnes pratiques sécurité : **moindre privilège**, conteneurs **non-root**, `resources.limits`, **NetworkPolicies**, scan d'images, mises à jour régulières.

3.5 Atelier — Déployer sur Minikube / K3s

🔧 Atelier 3 — Durée 2 h 30 — En binômes

Mission : déployer la QuickBite API sur un cluster local, l'exposer via Ingress, la configurer par ConfigMap/Secret, puis la mettre à l'échelle et tester un rollback.

Étapes : 1. (15 min) Démarrer le cluster : `minikube start` (activer l'ingress : `minikube addons enable ingress`). 2. (30 min) Appliquer `Deployment` + `Service` ; vérifier les Pods (`kubectl get pods`). 3. (25 min) Ajouter `ConfigMap` + `Secret` et les injecter. 4. (25 min) Exposer via `Ingress` ; ajouter l'hôte dans `/etc/hosts` (`minikube ip`), tester dans le navigateur. 5. (25 min) `kubectl scale` à 5 réplicas ; déployer une **mauvaise** image puis `rollout undo` . 6. (30 min) Restitution + debug (`describe` , `logs`).

Critères de réussite :

- 3 Pods `Running` et `Ready` .
- L'application répond via l'Ingress.
- Le rollback restaure la version saine.

⚠ Pièges fréquents à éviter

Anticipez : **image absente du cluster** (chargez-la avec `minikube image load` ou via un registre), **Ingress non prêt** (pensez à activer l'addon), **confusion Service / Ingress**. Votre premier réflexe de débogage doit toujours être `kubectl describe` + la lecture de la section *Events*.

Quiz — Jour 3

1. Qu'est-ce qu'un Pod ?
 2. Rôle d'un Deployment vs d'un Service ?
 3. Différence ConfigMap / Secret ?
 4. Que fait `kubectl rollout undo` ?
 5. Que signifie « approche déclarative » ?
-

Jour 4 — Automation & DevOps avancé

🎯 Objectifs du Jour 4

- Assembler une chaîne CI/CD complète : build → test → image → déploiement.
- Packager une application Kubernetes avec Helm.
- Mettre en place une observabilité de base (Prometheus + Grafana).
- Comprendre et appliquer le GitOps avec Argo CD.

4.1 Pipeline CI/CD de bout en bout

Objectif : à chaque tag, **construire l'image, la pousser, et déployer** sur Kubernetes.

```
name: CI-CD
on:
  push:
    tags: [ 'v*' ]
jobs:
  build-push:
    runs-on: ubuntu-latest
    permissions: { contents: read, packages: write }
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }
      - uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: ghcr.io/${ github.repository }:${ github.ref_name }
  deploy:
    needs: build-push
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Déployer sur Kubernetes
        run: |
          kubectl set image deploy/quickbite-api \
            api=ghcr.io/${ github.repository }:${ github.ref_name } \
            --namespace=prod
```

💡 Sécuriser les secrets de pipeline

N'écrivez jamais un token en clair : utilisez les **secrets** du dépôt (`${ secrets.X }`) et le principe du moindre privilège pour le compte de déploiement.

4.2 Helm Charts

Helm est le « gestionnaire de paquets » de Kubernetes : il *templatisé* les manifestes et les paramètre par environnement.

```
quickbite-chart/  
├─ Chart.yaml      # métadonnées du chart  
├─ values.yaml     # valeurs par défaut  
└─ templates/  
    ├─ deployment.yaml # avec des {{ .Values.* }}  
    ├─ service.yaml  
    └─ ingress.yaml
```

`values.yaml` :

```
replicaCount: 3  
image:  
  repository: ghcr.io/monuser/quickbite-api  
  tag: "1.0.0"  
service:  
  port: 80
```

Extrait de `templates/deployment.yaml` :

```
spec:  
  replicas: {{ .Values.replicaCount }}  
  template:  
    spec:  
      containers:  
        - name: api  
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
```

```
helm install quickbite ./quickbite-chart -n prod  
helm upgrade quickbite ./quickbite-chart --set image.tag=1.1.0 -n prod  
helm rollback quickbite 1 -n prod  
helm uninstall quickbite -n prod
```

Pourquoi Helm ?

Sans Helm, on duplique des YAML quasi identiques par environnement. Helm **factorise** (un template, plusieurs `values`) et fournit versioning + rollback applicatif.

4.3 Observabilité : Prometheus & Grafana

Les **3 piliers** : *Logs* (que s'est-il passé), *Métriques* (combien/à quelle vitesse), *Traces* (parcours d'une requête).

- **Prometheus** : collecte des **métriques** en *scrappant* des endpoints `/metrics` , les stocke en séries temporelles, et alerte (Alertmanager).
- **Grafana** : **visualise** ces métriques en tableaux de bord.

```

Application(/metrics) —scrape→ Prometheus —requête PromQL→ Grafana (dashboards)
                                   |
                                   → Alertmanager → email / Slack

```

Exemple de requête PromQL (taux d'erreurs HTTP 5xx) :

```

sum(rate(http_requests_total{status=~"5.."}[5m]))
/ sum(rate(http_requests_total[5m]))

```

💡 Les 4 signaux dorés (Google SRE)

À superviser en priorité : **Latence, Trafic, Erreurs, Saturation**. Un dashboard qui couvre ces quatre signaux suffit souvent à détecter 90 % des incidents.

4.4 GitOps avec Argo CD

GitOps : l'état souhaité du cluster vit dans **Git** (source de vérité unique). Un agent (**Argo CD**) compare en continu Git ↔ cluster et **synchronise** automatiquement.

```

Développeur → commit/PR → Dépôt Git (manifestes/Helm)
                                   |
                                   Argo CD surveille
                                   ▼
                                   Cluster Kubernetes (auto-sync vers l'état Git)

```

Avantages : **traçabilité** (tout passe par Git), **rollback** = `git revert`, **auditabilité**, fin des `kubectl apply` manuels en prod.

```

apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: { name: quickbite, namespace: argocd }
spec:
  project: default
  source:
    repoURL: https://github.com/monuser/quickbite-gitops
    targetRevision: main
    path: k8s/prod
  destination:
    server: https://kubernetes.default.svc
    namespace: prod
  syncPolicy:
    automated: { prune: true, selfHeal: true }

```

4.5 Atelier final — Chaîne complète CI/CD → Docker → Kubernetes

Atelier final — Durée 3 h — En équipes

Mission : assembler tout le module en une chaîne de bout en bout, du commit à la production locale.

Étapes : 1. (30 min) Le pipeline CI **teste** puis **construit et pousse** l'image (Jour 1 + Jour 2). 2. (40 min) Packager l'app en **Helm chart** paramétré (Jour 4). 3. (40 min) Déployer sur le cluster local via Helm ; exposer via Ingress (Jour 3). 4. (40 min) Brancher **Prometheus + Grafana** et afficher un dashboard (latence/erreurs). 5. (30 min, *bonus*) Mettre l'app sous **Argo CD** : un commit dans le repo GitOps déclenche le déploiement. 6. (20 min) Démonstration par équipe : « du commit à la prod ».

Critères de réussite (DoD) :

- Un push déclenche tests + build + push d'image (CI verte).
- `helm upgrade` déploie la nouvelle version sans coupure.
- Le dashboard Grafana affiche au moins un signal doré.
- (Bonus) Argo CD montre l'application *Synced / Healthy*.

La démonstration qui ancre tout

Quand votre chaîne fonctionne, faites l'exercice de la **raconter à voix haute**, « du commit à la prod » : à chaque étape, nommez l'outil et le garde-fou (test, revue, branche protégée, approbation). Le bonus Argo CD ne se tente qu'une fois le reste opérationnel. C'est cette narration complète qui ancre l'ensemble du module.

Partie pratique approfondie — Linux, Docker, Kubernetes, GitHub & CI/CD

■ De quoi avez-vous besoin pour suivre

Un environnement **Linux** : soit une machine ou une VM Ubuntu, soit **WSL2** sous Windows (`wsl --install`), soit un Mac (les commandes shell sont quasi identiques). Vous installerez au fil de l'eau : **Docker**, **kubectl** + un cluster local (**minikube** ou **kind**), **git**, le client **GitHub CLI** (`gh`) et un **JDK 21** + **Maven** pour QuickBite. Chaque atelier indique sa commande d'installation. Tapez les commandes vous-même : la mémoire des mains compte autant que celle de la tête.

Atelier Linux — le système, le terminal et le shell

Tout, en DevOps, finit sur **Linux** : vos conteneurs Docker tournent sur un noyau Linux, vos nœuds Kubernetes sont des machines Linux, les *runners* GitHub Actions par défaut sont des `ubuntu-latest`. Savoir se déplacer dans un terminal Linux n'est donc pas optionnel — c'est le socle de tout le reste.

🔗 Alternatives à connaître

Shell : `bash` est le standard pour les scripts ; `zsh` (par défaut sur macOS) et `fish` sont plus agréables en usage interactif. **Distributions** : la famille **Debian/Ubuntu** (gestionnaire `apt`) domine serveurs et CI ; **Fedora / RHEL / Rocky** utilisent `dnf` ; **Alpine** (gestionnaire `apk`), minuscule, équipe la plupart des images Docker. Les commandes vues ici sont communes à toutes.

L.1 Le shell, c'est quoi ?

Quand vous ouvrez un terminal, vous parlez à un programme appelé le **shell** (le plus courant est **bash**, un autre très répandu est **zsh**). Il lit ce que vous tapez, l'exécute, et vous rend la main. L'invite (le *prompt*) ressemble souvent à ceci :

```
haithem@laptop:~/projets/quickbite$  
|           |           |           |  
utilisateur machine dossier courant $ = utilisateur normal (# = root)
```

Le caractère `~` est un raccourci pour **votre dossier personnel** (`/home/haithem`). Le `$` final indique que vous êtes un utilisateur normal ; un `#` signifierait que vous êtes `root` (l'administrateur tout-puissant).

L.2 L'arborescence des fichiers

Sous Linux, **tout est un fichier** (même un périphérique ou un processus), et tout part d'une racine unique, `/`. Les dossiers que vous croiserez le plus :

Chemin	Contenu
<code>/</code>	la racine de tout
<code>/home/<user></code>	vos fichiers personnels
<code>/etc</code>	les fichiers de configuration du système
<code>/var</code>	les données variables : logs (<code>/var/log</code>), bases, files d'attente
<code>/tmp</code>	les fichiers temporaires (effacés au redémarrage)
<code>/usr/bin</code> , <code>/bin</code>	les programmes installés
<code>/opt</code>	les logiciels tiers installés à la main
<code>/proc</code> , <code>/sys</code>	des vues virtuelles du noyau et du matériel

Un **chemin absolu** commence par `/` (`/etc/hosts`) ; un **chemin relatif** part du dossier courant (`src/main`). Deux raccourcis essentiels : `.` = le dossier courant, `..` = le dossier parent.

L.3 Se déplacer et manipuler des fichiers

```

pwd                # "print working directory" : où suis-je ?
ls                 # lister le contenu du dossier
ls -la            # -l = détaillé, -a = inclut les fichiers cachés (.xxx)
cd /etc           # aller dans /etc
cd ..             # remonter d'un cran
cd                # revenir dans son dossier personnel (~)
cd -              # revenir au dossier précédent

mkdir -p projets/quickbite # créer une arborescence (-p crée les parents)
touch notes.txt           # créer un fichier vide (ou actualiser sa date)
cp notes.txt sauvegarde.txt # copier
cp -r dossier/ copie/      # copier un dossier entier (-r = récursif)
mv notes.txt docs/         # déplacer (ou renommer)
rm sauvegarde.txt          # supprimer un fichier
rm -r copie/              # supprimer un dossier et son contenu

```

! **rm -rf ne pardonne pas**

Il n'y a **pas de corbeille** en ligne de commande. `rm -rf /chemin` efface définitivement et sans confirmation. Une faute de frappe comme un espace de trop (`rm -rf / tmp` au lieu de `rm -rf /tmp`) peut détruire le système. Relisez toujours une commande `rm -rf` **avant** d'appuyer sur Entrée, et préférez des chemins absolus explicites.

L.4 Lire le contenu des fichiers

```
cat application.yml          # afficher tout le fichier d'un coup
less /var/log/syslog         # parcourir un gros fichier (q pour quitter, / pour chercher)
head -n 20 fichier.log       # les 20 premières lignes
tail -n 50 fichier.log       # les 50 dernières lignes
tail -f /var/log/app.log     # SUIVRE en direct (idéal pour observer des logs)
wc -l fichier.log            # compter les lignes
```

`tail -f` (« follow ») est un réflexe à acquérir : il affiche les nouvelles lignes au fur et à mesure qu'elles sont écrites. C'est exactement ce que fait `docker logs -f` ou `kubectl logs -f`.

L.5 Chercher : `grep`, `find`

```
grep "ERROR" app.log         # toutes les lignes contenant ERROR
grep -i "error" app.log      # -i = insensible à la casse
grep -r "TODO" src/          # -r = chercher récursivement dans une arbo
grep -n "port" application.yml # -n = afficher les numéros de ligne
grep -c "200" access.log     # -c = compter les occurrences

find . -name "*.java"        # tous les fichiers .java sous le dossier courant
find . -type d -name target  # tous les DOSSIERS (-type d) nommés target
find /var/log -mtime -1      # fichiers modifiés il y a moins d'un jour
```

L.6 Les tuyaux (`|`) et les redirections — la vraie puissance d'Unix

La philosophie Unix : de **petits outils** qui font **une seule chose**, qu'on **enchaîne**. Le « tuyau » `|` envoie la sortie d'une commande comme entrée de la suivante.

```
cat access.log | grep "500" | wc -l    # combien d'erreurs 500 dans le log ?
ps aux | grep java                    # tous les processus liés à java
ls -la | sort -k5 -n                  # trier les fichiers par taille

commande > sortie.txt                 # rediriger la sortie vers un fichier (écrase)
commande >> sortie.txt                 # ajouter à la fin du fichier (n'écasse pas)
commande 2> erreurs.txt               # rediriger les ERREURS (flux 2) vers un fichier
commande > tout.txt 2>&1               # rediriger sortie ET erreurs au même endroit
commande < entree.txt                 # lire l'entrée depuis un fichier
```

Et les enchaînements conditionnels, omniprésents dans les scripts CI/CD :

```
mvn test && docker build -t app .      # build SEULEMENT si les tests passent (&&)
ping -c1 serveur || echo "injoignable" # message SI la commande échoue (||)
```

L.7 Traiter du texte : cut, sort, uniq, sed, awk

```
cut -d',' -f1,3 data.csv           # extraire les colonnes 1 et 3 (séparateur ,)
sort fichier.txt | uniq -c          # compter les occurrences de chaque ligne
sed 's/dev/prod/g' config.txt      # remplacer "dev" par "prod" (g = partout)
awk '{print $1}' access.log         # afficher la 1re colonne de chaque ligne
awk '{print $1}' access.log | sort | uniq -c | sort -rn # top des IP
```

Pas besoin de tout maîtriser : retenez que `grep` filtre, `cut` / `awk` découpent en colonnes, `sed` remplace, `sort` / `uniq` agrègent. Avec ces briques on analyse n'importe quel log.

L.8 Les permissions : qui a le droit de quoi

Chaque fichier appartient à un **utilisateur** et à un **groupe**, et porte trois niveaux de droits : **read** (lire), **write** (écrire), **execute** (exécuter). `ls -l` les affiche :

```
-rwxr-xr-- 1 haithem devs 4096 app.sh
```

├───┬───┬───┬───┐
│ │ │ │ │
└───┴───┴───┘ autres : r-- (lecture seule)
└──────────┘ groupe : r-x (lecture + exécution)
└──────────┘ proprio : rwx (tout)
└──────────┘ type : - = fichier, d = dossier, l = lien

On modifie les droits avec `chmod`, en notation **numérique** (r=4, w=2, x=1, qu'on additionne) ou **symbolique** :

```
chmod 755 app.sh      # rwx pour le proprio (7), r-x pour groupe et autres (5)
chmod 644 config.yml  # rw- pour le proprio, r-- pour les autres
chmod +x deploy.sh    # rendre un script exécutable (forme symbolique)
chown haithem:devs app.sh # changer le propriétaire et le groupe
```

Le mot **sudo** (« superuser do ») exécute **une seule commande** avec les droits d'administrateur — c'est plus sûr que de se connecter en **root** en permanence :

```
sudo apt update          # une action administrateur ponctuelle
sudo systemctl restart nginx
```

💡 « Permission denied » : le réflexe

Quand une commande échoue avec **Permission denied** , ne préfixez pas aveuglément par **sudo** . Demandez-vous d'abord **pourquoi** : un script sans **+x** ? un fichier appartenant à un autre utilisateur ? un port < 1024 réservé ? Comprendre la cause évite d'accorder des droits trop larges « pour que ça marche ».

L.9 Les processus

Un **processus** est un programme en cours d'exécution. Le système leur attribue un identifiant, le **PID**.

```
ps aux          # lister TOUS les processus (a=tous, u=détaillé, x=arrière-plan)
ps aux | grep java # ne garder que ceux liés à java
top             # moniteur temps réel (CPU, mémoire) – q pour quitter
htop           # version plus lisible (à installer : sudo apt install htop)

kill 1234       # demander poliment au processus 1234 de s'arrêter (SIGTERM)
kill -9 1234    # le forcer brutalement (SIGKILL) – en dernier recours
pkill -f quickbite # tuer par nom/motif de commande
```

Lancer un programme en **arrière-plan** et le garder en vie :

```
./serveur.sh &          # & = lancer en arrière-plan
nohup ./serveur.sh &    # nohup = survit à la fermeture du terminal
jobs                    # voir les tâches d'arrière-plan du shell
```

La distinction **SIGTERM** (15, « range tes affaires et arrête-toi ») vs **SIGKILL** (9, « arrêt immédiat sans préavis ») est importante : c'est exactement ce que fait Docker quand il arrête un conteneur (**stop** envoie SIGTERM puis SIGKILL après un délai de grâce).

L.10 Installer des logiciels (gestionnaire de paquets)

Sur les distributions Debian/Ubuntu, le gestionnaire est **apt** :

```
sudo apt update          # rafraîchir la liste des paquets disponibles
sudo apt install -y htop git curl # installer des paquets (-y = sans confirmation)
sudo apt upgrade         # mettre à jour les paquets installés
sudo apt remove htop     # désinstaller
apt search docker        # chercher un paquet
```

Sur Red Hat/Fedora/CentOS, c'est **dnf** (ou l'ancien **yum**) avec la même logique : **sudo dnf install ...**. **À retenir** : sur Debian/Ubuntu, **apt update** ne **met pas à jour** les logiciels — il met à jour la **liste** ; c'est **apt upgrade** qui installe les nouvelles versions.

L.11 Les services et **systemd**

Un **service** (ou *daemon*) est un programme qui tourne en permanence en arrière-plan (un serveur web, une base de données...). Sur Linux moderne, c'est **systemd** qui les gère, via la commande **systemctl** :

```
sudo systemctl status docker # l'état d'un service (actif ? depuis quand ?)
sudo systemctl start docker  # démarrer
sudo systemctl stop docker   # arrêter
sudo systemctl restart docker # redémarrer
sudo systemctl enable docker # le lancer AUTOMATIQUEMENT au démarrage de la machine
sudo systemctl disable docker # ne plus le lancer au démarrage

journalctl -u docker          # les logs du service docker
journalctl -u docker -f       # les suivre en direct
journalctl -u docker --since "10 min ago"
```

La nuance `start` vs `enable` revient souvent : `start` lance **maintenant** ; `enable` fait en sorte qu'il se relance **tout seul après un redémarrage**. On combine généralement les deux : `sudo systemctl enable --now docker`.

L.12 Le réseau

```
ip a          # afficher les interfaces réseau et leurs adresses IP
ss -tulpn     # les ports en écoute (t=tcp, u=udp, l=listen, p=process, n=numérique)
ping -c4 google.com # tester la connectivité (4 paquets)
curl -i http://localhost:8080/health # appeler une URL HTTP et voir les en-têtes (-i)
curl -s http://localhost:8080/actuator/health | jq # -s silencieux, jq formate le JSON
```

Pour se connecter à une machine distante et y copier des fichiers :

```
ssh haithem@192.168.1.10 # ouvrir une session distante
scp app.jar haithem@serveur:/opt/ # copier un fichier vers la machine distante
```

`ss -tulpn` est l'outil n°1 quand un service « ne répond pas » : il dit **quel programme écoute sur quel port**. Si votre `:8080` n'apparaît pas, l'application n'est pas démarrée ou écoute ailleurs.

L.13 Variables d'environnement

Une **variable d'environnement** est une valeur nommée que le système transmet aux programmes. C'est le moyen standard de configurer une application **sans toucher à son code** (et c'est central en Docker/Kubernetes) :

```
echo $HOME          # afficher une variable existante
echo $PATH           # la liste des dossiers où le shell cherche les programmes
export SPRING_PROFILES_ACTIVE=postgres # définir une variable pour la session
export DATABASE_URL="jdbc:postgresql://db:5432/quickbite"
printenv             # lister toutes les variables d'environnement
```

Le `PATH` mérite une explication : quand vous tapez `docker`, le shell cherche un programme nommé `docker` dans chacun des dossiers listés dans `$PATH`, dans l'ordre. C'est pourquoi un outil « introuvable » l'est souvent simplement parce que son dossier n'est pas dans le `PATH`. Pour rendre une variable permanente, on l'ajoute au fichier `~/.bashrc` (ou `~/.zshrc`), lu à chaque ouverture de terminal.

L.14 Écrire un script shell

Un **script** est une suite de commandes dans un fichier, qu'on exécute d'un coup. C'est la base de l'automatisation (et une étape de pipeline CI/CD n'est souvent rien d'autre qu'un script). Créez `deploy.sh` :

```
#!/usr/bin/env bash
# La première ligne (le "shebang") indique avec quel interpréteur exécuter le fichier.

set -euo pipefail
# -e : arrêter au premier échec. -u : erreur si une variable est indéfinie.
# -o pipefail : un échec au milieu d'un | fait échouer toute la chaîne.

APP_NAME="quickbite-api"           # une variable
VERSION="${1:-latest}"             # $1 = 1er argument ; "latest" par défaut

echo "► Construction de ${APP_NAME}:${VERSION}"

if [ -f "pom.xml" ]; then           # tester l'existence d'un fichier
    mvn -B clean package -DskipTests
else
    echo "Erreur : pom.xml introuvable" >&2 # écrire sur la sortie d'erreur
    exit 1                                # quitter avec un code d'échec
fi

for env in dev staging prod; do     # une boucle
    echo " → préparation de l'environnement ${env}"
done

echo "✅ Terminé."
```

On le rend exécutable puis on le lance :

```
chmod +x deploy.sh
./deploy.sh 1.2.0           # "1.2.0" devient $1
echo $?                     # afficher le CODE DE SORTIE de la dernière commande (0 = succès)
```

Le **code de sortie** (`exit code`) est fondamental : par convention, **0 = succès**, tout autre nombre = échec. C'est sur ce code que reposent `&&`, `||` et **toute** la logique des pipelines CI : une étape « rouge », c'est simplement une commande qui a renvoyé un code différent de 0.

Exercice — Atelier Linux

Sur votre machine Linux/WSL :

1. Créez l'arborescence `~/labs/quickbite/{src,logs,config}` en **une** commande `mkdir -p`.
2. Générez un faux log : `for i in $(seq 1 100); do echo "$i $((RANDOM%500+100)) /api/orders"; done > logs/access.log`.
3. Avec un **tuyau**, comptez combien de lignes contiennent un code `500`, puis affichez le **top 3** des codes les plus fréquents (`awk + sort + uniq -c`).
4. Écrivez un script `healthcheck.sh` qui appelle `curl -s http://localhost:8080/health` et affiche « OK » ou « KO » selon le **code de sortie** de `curl`, en utilisant `set -euo pipefail`.
5. Rendez-le exécutable et vérifiez `echo $?`.

Objectif : être à l'aise pour naviguer, filtrer des logs et écrire un script simple — ce sont les gestes que vous referez dans chaque atelier suivant.

Atelier API REST & documentation (OpenAPI / Swagger)

QuickBite expose une **API REST**. Avant de la conteneuriser et de la déployer, encore faut-il qu'elle soit **bien conçue** et **documentée** — sinon personne (ni le frontend, ni les autres équipes, ni vous-même dans six mois) ne saura s'en servir.

🔗 Alternatives à connaître

Documentation Spring : `springdoc-openapi` est le choix actuel (l'ancien **Springfox** n'est plus maintenu). **Rendu de la doc** : **Swagger UI** (interactif) ou **Redoc** (lecture élégante). **Contract-first** : **Swagger Editor**, **Stoplight**, **Apicurio**. **Clients de test** : **Postman** et **Insomnia** importent directement votre spec OpenAPI. **Autres styles d'API** : **GraphQL** et **gRPC**.

R.1 Les principes REST en cinq idées

REST organise une API autour de **ressources** (des noms : `orders`, `users`, `menu-items`) que l'on manipule avec les **verbes HTTP** :

Verbe	Intention	Exemple	Idempotent ?
GET	lire	GET /orders/42	oui
POST	créer	POST /orders	non
PUT	remplacer entièrement	PUT /orders/42	oui
PATCH	modifier partiellement	PATCH /orders/42	non
DELETE	supprimer	DELETE /orders/42	oui

Trois règles de bon sens pour les URL :

- des **noms au pluriel**, jamais de verbes : `GET /orders`, et surtout pas `GET /getOrders` ;
- la **hiérarchie** exprime l'appartenance : `GET /orders/42/items` ;
- le **filtrage, le tri et la pagination** passent par la *query string* : `GET /orders?status=PAID&page=2&size=20&sort=createdAt,desc`.

R.2 Les codes de statut HTTP : parler le bon langage

Le code de statut est la **première information** que reçoit le client. Le choisir correctement est une marque de qualité :

Code	Sens	Quand
200 OK	succès	lecture ou mise à jour réussie
201 Created	ressource créée	après un POST réussi
204 No Content	succès sans corps	après un DELETE
400 Bad Request	requête invalide	corps malformé, validation échouée
401 Unauthorized	non authentifié	token absent ou invalide
403 Forbidden	authentifié mais pas autorisé	rôle insuffisant
404 Not Found	ressource inexistante	GET /orders/99999
409 Conflict	conflit d'état	doublon, version périmée
422 Unprocessable	sémantiquement invalide	règle métier non respectée
429 Too Many Requests	trop d'appels	rate limiting
500 Internal Server Error	bug serveur	exception non gérée

La distinction **401** (« je ne sais pas qui tu es ») vs **403** (« je sais qui tu es, mais tu n'as pas le droit ») rejoint exactement l'authentification/autorisation du Module 3.

R.3 Des erreurs cohérentes (RFC 7807)

Une bonne API renvoie ses erreurs dans un format **uniforme et lisible par une machine**. Le standard est `application/problem+json` (RFC 7807), supporté nativement par Spring 6 via `ProblemDetail` :

```
@ExceptionHandler(OrderNotFoundException.class)
ProblemDetail handle(OrderNotFoundException ex) {
    var pd = ProblemDetail.forStatus(HttpStatus.NOT_FOUND);
    pd.setTitle("Commande introuvable");
    pd.setDetail(ex.getMessage()); // message clair, SANS détail technique sensible
    return pd;
}
```

R.4 Versionner l'API

On préfixe les chemins par une version (`/api/v1/orders`) afin de pouvoir faire **évoluer** le contrat (ajouter un champ, changer un format) sans **casser** les clients déjà en place, qui restent sur `v1` tant qu'ils n'ont pas migré.

R.5 Documenter automatiquement avec OpenAPI

OpenAPI (l'ex-**Swagger**) est un format standard qui décrit une API : ses chemins, ses paramètres, ses schémas de données, ses codes de réponse. Plutôt que de l'écrire à la main, on le **génère depuis le code** avec `springdoc-openapi`. Ajoutez la dépendance au `pom.xml` :


```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.6.0</version>
</dependency>
```

Au démarrage de QuickBite, deux URL apparaissent automatiquement :

- `/v3/api-docs` : la spécification OpenAPI au format JSON (consommable par d'autres outils : génération de clients, tests de contrat, Postman) ;
- `/swagger-ui.html` : une **interface web interactive** pour explorer **et tester** l'API directement depuis le navigateur.

On enrichit la documentation avec quelques annotations sur les contrôleurs :

```
@Operation(summary = "Créer une commande", description = "Crée une commande pour l'utilisateur courant")
@ApiResponse(responseCode = "201", description = "Commande créée")
@ApiResponse(responseCode = "400", description = "Requête invalide")
@PostMapping("/api/v1/orders")
public ResponseEntity<OrderDto> create(@Valid @RequestBody CreateOrder body) { ... }
```

⚠️ Swagger derrière Spring Security

QuickBite étant sécurisé (Module 3), les chemins `/swagger-ui/**` et `/v3/api-docs/**` seront **bloqués** par défaut. Pensez à les autoriser explicitement dans votre `SecurityConfig` (en environnement de dev), au même titre que `/health`. Et n'exposez jamais Swagger UI **en production** sans authentification : c'est une cartographie complète de votre API offerte à un attaquant.

💡 Code-first ou contract-first ?

Deux écoles. En **code-first** (ce qu'on vient de faire), la doc est générée *depuis* le code : simple et toujours synchronisée. En **contract-first**, on écrit d'abord le fichier OpenAPI, validé avec le frontend, puis on génère le squelette du serveur et des clients : idéal quand plusieurs équipes doivent se mettre d'accord **avant** de coder. Pour QuickBite, le code-first suffit ; retenez juste que le contract-first existe et brille en contexte multi-équipes.

Exercice — Atelier API REST & OpenAPI

1. Ajoutez `springdoc-openapi` au `pom.xml` de QuickBite et autorisez `/swagger-ui/**` + `/v3/api-docs/**` dans la configuration de sécurité.
2. Démarrez l'application et ouvrez `/swagger-ui.html` ; explorez les endpoints existants (auth, orders).
3. **Testez une création de commande** directement depuis Swagger UI et vérifiez que le code renvoyé est bien `201`.
4. Ajoutez un `@ExceptionHandler` qui renvoie un `ProblemDetail` (RFC 7807) pour une commande introuvable, et vérifiez le `404` propre.
5. Récupérez le JSON `/v3/api-docs` et importez-le dans Postman pour générer la collection automatiquement.

Objectif : une API dont le contrat est explicite, testable en un clic et toujours à jour.

Atelier Tests automatisés — le filet qui rend la CI utile

Toute la mécanique CI/CD repose sur une promesse : « si les tests passent, on peut livrer ». Encore faut-il **avoir** de bons tests. Cet atelier comble le maillon le plus souvent négligé.

Alternatives à connaître

Framework : **JUnit 5** est le standard Java ; **TestNG** en est l'alternative historique.
Assertions : **AssertJ** (fluide, recommandé) ou **Hamcrest**. **Mocks :** **Mockito** domine ; **EasyMock** existe. **Test d'API :** **REST Assured**. **Simulation de services HTTP :** **WireMock**.
End-to-end (navigateur) : **Selenium**, **Playwright** ou **Cypress**.

T.1 La pyramide des tests



end-to-end (e2e) : peu, lents, fragiles, réalistes

intégration : moyens, vérifient l'assemblage

unitaires : beaucoup, rapides, isolés

Le principe : **beaucoup** de petits tests unitaires (millisecondes, exécutés à chaque sauvegarde), **moins** de tests d'intégration, et **très peu** de tests end-to-end (lents et coûteux à maintenir). Une pyramide inversée — quelques gros tests e2e et rien d'autre — donne une suite lente et instable.

T.2 Le test unitaire (JUnit 5)

Un test unitaire vérifie **une seule unité** (une méthode, une classe) **en isolation**. On suit la structure **AAA** (*Arrange - Act - Assert*) :

```
class PriceCalculatorTest {

    @Test
    void applique_une_remise_de_10_pourcent() {
        var calc = new PriceCalculator();           // Arrange : on prépare
        var total = calc.withDiscount(100, 0.10);    // Act : on exécute
        assertThat(total).isEqualTo(90);            // Assert : on vérifie
    }
}
```

```
mvn test # lance tous les tests unitaires
```

Nommez vos tests pour qu'ils **décrivent un comportement** (`refuse_une_commande_vide`), pas la méthode appelée. Un test qui échoue doit dire, par son nom, **ce qui** est cassé.

T.3 Isoler avec des mocks (Mockito)

Quand l'unité testée dépend d'autre chose (une base, un service externe), on **simule** cette dépendance avec un **mock** pour rester rapide et isolé :

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

    @Mock OrderRepository repo;           // une fausse base, contrôlée par le test
    @InjectMocks OrderService service;    // le service réel, avec le mock injecté

    @Test
    void refuse_une_commande_sans_article() {
        assertThatThrownBy(() -> service.create(new Order(List.of()))
            .isInstanceOf(InvalidOrderException.class);
        verify(repo, never()).save(any()); // on vérifie qu'on n'a RIEN enregistré
    }
}
```

Le mock permet de **provoquer** des situations difficiles à reproduire (une panne réseau, une réponse précise) et de **vérifier les interactions** (`verify`).

T.4 Le test d'intégration Spring Boot

Ici, on charge une (partie de l') application réelle. Avec **MockMvc**, on teste la couche web de bout en bout, **sans démarrer de vrai serveur** :

```
@SpringBootTest
@AutoConfigureMockMvc
class OrderControllerIT {

    @Autowired MockMvc mvc;

    @Test
    void cree_une_commande_et_renvie_201() throws Exception {
        mvc.perform(post("/api/v1/orders")
            .contentType(MediaType.APPLICATION_JSON)
            .content("{\"item\":\"pizza\",\"qty\":2}"))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.id").exists());
    }
}
```

T.5 Testcontainers : une vraie base, jetable

Tester contre H2 (en mémoire) ne garantit pas que tout fonctionnera avec **PostgreSQL** en production (dialectes SQL différents). **Testcontainers** démarre une **vraie base PostgreSQL dans un conteneur Docker** le temps des tests, puis la détruit :

```
@SpringBootTest
@Testcontainers
class OrderRepositoryIT {

    @Container
    static PostgreSQLContainer<?> db = new PostgreSQLContainer<>("postgres:16-alpine");

    @DynamicPropertySource
    static void props(DynamicPropertyRegistry r) {
        r.add("spring.datasource.url", db::getJdbcUrl);
        r.add("spring.datasource.username", db::getUsername);
        r.add("spring.datasource.password", db::getPassword);
    }

    // ... les tests s'exécutent désormais contre un PostgreSQL RÉEL
}
```

💡 Testcontainers tourne aussi en CI

Comme les *runners* `ubuntu-latest` de GitHub Actions ont Docker préinstallé, vos tests Testcontainers s'exécutent **tels quels** dans le pipeline. Vous testez donc contre la même base qu'en production, à chaque push, sans rien installer de spécial. C'est la fin du « ça passe en local mais pas en CI ».

T.6 Mesurer la couverture (JaCoCo)

La **couverture de code** indique le pourcentage de lignes exécutées par les tests. On l'active avec le plugin **JaCoCo** dans le `pom.xml` ; le rapport est généré par `mvn verify`.

⚠ 100 % de couverture ≠ zéro bug

La couverture mesure ce que vos tests **traversent**, pas ce qu'ils **vérifient** : on peut exécuter une ligne sans tester son résultat. Un chiffre élevé obtenu avec des tests sans assertions est un faux sentiment de sécurité. Utilisez la couverture pour repérer le code **jamais testé** (souvent le plus risqué), pas comme un objectif chiffré à atteindre coûte que coûte.

T.7 Le réflexe TDD (en deux mots)

Le **TDD** (*Test-Driven Development*) inverse l'ordre habituel : **Red → Green → Refactor**. On écrit d'abord un test qui échoue (red), on écrit le minimum de code pour le faire passer (green), puis on nettoie (refactor). On code ainsi exactement ce qui est nécessaire, avec un filet dès la première ligne.

T.8 Le lien direct avec la CI

Par convention Maven, `mvn verify` lance **les deux** familles : les tests unitaires (classes en `*Test`, plugin `surefire`) **et** les tests d'intégration (classes en `*IT`, plugin `failsafe`). C'est **exactement** la commande qu'exécute le workflow `ci.yml` de l'atelier GitHub Actions. Écrire de bons tests, c'est donc directement renforcer le garde-fou de votre pipeline.

✏ Exercice — Atelier Tests

Sur QuickBite :

1. Écrivez un **test unitaire** (JUnit 5 + AssertJ) sur une règle métier (ex. calcul du total d'une commande).
2. Écrivez un test de service avec **Mockito** qui vérifie qu'une commande invalide est **refusée** et **jamais enregistrée**.
3. Écrivez un test d'**intégration** `OrderControllerIT` avec MockMvc qui valide le `201` à la création.
4. Ajoutez un test **Testcontainers** contre un vrai PostgreSQL.
5. Lancez `mvn verify` en local, puis vérifiez que le **même** `verify` passe dans votre pipeline GitHub Actions.

Critère de réussite : casser volontairement une règle métier doit faire échouer `mvn verify` **et** rougir la CI — donc bloquer la fusion.

Atelier Migrations de base de données (Flyway)

Le schéma de la base **évolue** avec l'application : on ajoute une table, une colonne, un index. Comment versionner ces changements et les appliquer de façon **fiable, automatique et reproductible** sur chaque environnement (dev, CI, production) ? C'est le rôle des outils de **migration**. On utilise **Flyway** (l'alternative étant Liquibase).

✂ Alternatives à connaître

Flyway : simple, fondé sur du **SQL pur**, prise en main rapide — idéal pour apprendre et pour la plupart des projets. **Liquibase** : plus puissant et plus abstrait (changelogs en XML/YAML/JSON/SQL, *rollback* automatique, portabilité multi-SGBD), au prix d'une courbe d'apprentissage plus raide. Pour QuickBite, Flyway suffit et reste parfaitement lisible.

F.1 Le problème : appliquer les changements « à la main »

Sans outil, on applique les `ALTER TABLE` manuellement sur chaque base. Résultat : les environnements **divergent** (une colonne existe en dev mais pas en prod), il n'y a **aucune trace** de qui a appliqué quoi, et il devient **impossible** de recréer une base propre à l'identique. En production, c'est une source d'incidents majeurs.

F.2 Le principe de Flyway

Flyway applique des **scripts SQL versionnés, immuables**, exécutés **dans l'ordre** et **une seule fois** chacun. Il tient une table de suivi `flyway_schema_history` qui mémorise les migrations déjà passées (avec une **somme de contrôle**). La convention de nommage est stricte : `V1__creation_table_orders.sql`, `V2__ajout_colonne.sql`, etc. (deux underscores après le numéro de version).

F.3 Mise en place dans Spring Boot

```
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-core</artifactId>
</dependency>
<dependency>                                <!-- support spécifique PostgreSQL -->
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-database-postgresql</artifactId>
</dependency>
```

Spring Boot exécute Flyway **automatiquement au démarrage** de l'application : les migrations en attente sont appliquées avant que l'API ne réponde. Les scripts se placent dans `src/main/resources/db/migration/`.

F.4 Écrire des migrations

`V1__create_orders.sql` :

```
CREATE TABLE orders (
  id          BIGSERIAL PRIMARY KEY,
  item        VARCHAR(120) NOT NULL,
  qty         INT          NOT NULL CHECK (qty > 0),
  status      VARCHAR(20)  NOT NULL DEFAULT 'PENDING',
  created_at  TIMESTAMP    NOT NULL DEFAULT now()
);
```

V2__add_customer_to_orders.sql :

```
ALTER TABLE orders ADD COLUMN customer_email VARCHAR(160);  
CREATE INDEX idx_orders_status ON orders(status);
```

Au démarrage, Flyway constate que **V1** et **V2** ne sont pas encore appliquées, les exécute dans l'ordre, et enregistre chacune dans **flyway_schema_history**. Au prochain démarrage, il ne fait **rien** (idempotence) — elles sont déjà passées.

! Une migration appliquée est IMMuable

Ne modifiez **jamais** un script **V** déjà appliqué : Flyway recalcule sa somme de contrôle au démarrage et **refuse de démarrer** s'il a changé (c'est une protection, pas un bug). Pour corriger une erreur, on crée une **nouvelle** migration (**V3**) qui rectifie. La règle d'or des bases en production : **on n'édite jamais le passé, on ajoute**.

F.5 Les commandes utiles

```
mvn flyway:info      # l'état des migrations (appliquées / en attente)  
mvn flyway:migrate   # applique les migrations en attente  
mvn flyway:validate  # vérifie l'intégrité (sommes de contrôle)  
mvn flyway:repair     # répare la table d'historique après un échec
```

F.6 Migrations et CI/CD

- **En test** : couplées à **Testcontainers** (atelier précédent), les migrations s'exécutent sur une vraie base PostgreSQL jetable — vous testez donc le **schéma réel**, pas une approximation.
- **Au déploiement** : on applique les migrations **avant** de démarrer la nouvelle version de l'application.

⚠ Migrations et déploiement sans coupure

Lors d'un *rolling update* Kubernetes, l'**ancienne** et la **nouvelle** version de l'application tournent un court instant **en même temps**. Une migration destructive (**DROP COLUMN**) appliquée immédiatement casserait l'ancienne version encore en service. La bonne pratique est la migration en **deux temps** (« expand / contract ») : d'abord **ajouter** ce qui est compatible et déployer, puis, une fois l'ancienne version totalement retirée, faire une seconde migration qui **supprime** l'ancien. Jamais les deux d'un coup.

Exercice — Atelier Flyway

Sur QuickBite :

1. Ajoutez `flyway-core` au `pom.xml` et créez `V1__create_orders.sql`.
2. Démarrez l'application : vérifiez dans les logs que Flyway applique `V1`, puis inspectez la table `flyway_schema_history`.
3. Ajoutez `V2__add_customer_to_orders.sql`, redémarrez, et constatez que **seule** `V2` est appliquée.
4. **Tentez de modifier** `V1` déjà appliquée et observez Flyway **refuser de démarrer** (somme de contrôle). Corrigez la situation avec une migration `V3`.
5. Branchez les migrations sur votre test **Testcontainers** : le schéma testé devient le schéma réel.

Objectif : que le schéma de base soit versionné dans Git, rejouable à l'identique partout, et validé par la CI.

Atelier Docker pratique — de l'image au conteneur

Le Jour 2 a expliqué **pourquoi** Docker. Place à la pratique : on installe, on lance, on construit, on débogue, et on conteneurise réellement QuickBite.

Alternatives à connaître

Podman : compatible avec la CLI Docker (`alias docker=podman`), **sans démon** et **rootless** par défaut (plus sûr). **containerd + nerdctl** : le moteur de conteneurs qui tourne *sous* Docker et Kubernetes. **Buildah / Kaniko** : construire des images **sans** Docker (pratique en CI). Le format étant standardisé (**OCI**), une image construite par l'un fonctionne avec les autres. Docker reste la référence sur le poste de développement.

D.1 Installer et vérifier

```
# Ubuntu : script officiel (pour un poste de dev)
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER          # pour utiliser docker sans sudo (se reconnecter ensuite)

docker version                         # versions client + serveur
docker run hello-world                 # le "hello world" : télécharge et lance une image test
docker info                            # état du démon, nombre d'images/conteneurs
```

Si `hello-world` affiche son message, votre installation fonctionne : Docker a **téléchargé** une image depuis Docker Hub, **créé** un conteneur, l'a **exécuté**, puis il s'est arrêté.

D.2 Le cycle de vie d'un conteneur

```
docker run -d -p 8080:80 --name web nginx    # -d détaché, -p publie le port, --name le nomme
docker ps                                   # les conteneurs EN COURS
docker ps -a                               # TOUS, y compris arrêtés
docker logs web                             # les logs du conteneur
docker logs -f web                         # les suivre en direct
docker exec -it web bash                   # ouvrir un shell DANS le conteneur (-it interactif)
docker stop web                            # arrêt propre (SIGTERM puis SIGKILL)
docker start web                            # le relancer
docker rm -f web                           # le supprimer (-f force même s'il tourne)
```

Ouvrez `http://localhost:8080` : la page d'accueil Nginx s'affiche, servie depuis le conteneur. La commande `docker exec -it web bash` est votre meilleur ami pour **inspecter de l'intérieur** un conteneur qui se comporte mal.

D.3 Images : lister, inspecter, nettoyer

```
docker images                               # les images locales
docker pull postgres:16-alpine              # télécharger une image sans la lancer
docker inspect nginx                        # toutes les métadonnées (JSON)
docker history monimage:1.0                 # voir les COUCHES qui composent l'image
docker rmi nginx                            # supprimer une image

docker system df                             # combien d'espace Docker occupe
docker system prune -a                       # GRAND nettoyage : images/conteneurs/réseaux inutilisés
```

💡 Les couches et le cache, vus en vrai

Lancez `docker history` sur une de vos images : chaque ligne est une **couche**, créée par une instruction du Dockerfile. Docker **réutilise** une couche tant que ce qui la précède n'a pas changé. C'est la raison pour laquelle on copie `pom.xml` (ou `package.json`) **avant** le code source : tant que les dépendances ne bougent pas, la couche `mvn dependency:go-offline` reste en cache et le rebuild prend quelques secondes au lieu de plusieurs minutes.

D.4 Construire l'image de QuickBite (Spring Boot, multi-stage)

QuickBite est une application **Spring Boot / Maven / Java 21**. Voici le `Dockerfile` multi-stage, commenté — c'est exactement le motif utilisé en production pour une appli Java :

```
# ----- Étape 1 : build (avec Maven + JDK) -----
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn -B dependency:go-offline      # couche STABLE : ne se reconstruit que si pom.xml change
COPY src ./src
RUN mvn -B clean package -DskipTests # produit target/quickbite-api-1.0.0.jar

# ----- Étape 2 : runtime (JRE seule, image légère) -----
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
ENV SPRING_PROFILES_ACTIVE=postgres
COPY --from=build /app/target/quickbite-api-*.jar app.jar # on ne récupère QUE le .jar
EXPOSE 8080
RUN addgroup -S app && adduser -S app -G app # créer un utilisateur non-root
USER app # ne jamais tourner en root
ENTRYPOINT ["java", "-jar", "app.jar"]
```

On construit puis on lance :

```
docker build -t quickbite-api:1.0.0 . # construire l'image (le . = contexte)
docker images | grep quickbite        # vérifier sa taille (~200 Mo grâce à la JRE alpine)
docker run -d -p 8080:8080 --name qb quickbite-api:1.0.0
curl http://localhost:8080/health     # tester l'endpoint de santé
docker logs -f qb                    # observer le démarrage de Spring
```

L'image finale n'embarque **ni Maven ni le JDK complet** : seulement une **JRE** + le **.jar**. C'est tout l'intérêt du multi-stage : une image plus **petite**, plus **rapide** à télécharger, et avec **moins de surface d'attaque**.

D.5 Variables d'environnement et configuration

On ne reconstruit pas une image pour changer un paramètre : on injecte la configuration **au lancement**. Spring lit naturellement les variables d'environnement :

```
docker run -d -p 8080:8080 \
  -e SPRING_PROFILES_ACTIVE=postgres \
  -e SPRING_DATASOURCE_URL="jdbc:postgresql://db:5432/quickbite" \
  -e SPRING_DATASOURCE_PASSWORD="secret" \
  --name qb quickbite-api:1.0.0
```

! Le mot de passe en clair dans l'historique

Passer un secret avec `-e MOT_DE_PASSE=...` le rend visible dans `docker inspect` et dans l'historique de votre shell (`history`). En production, on utilise un fichier `--env-file` non versionné, ou un vrai gestionnaire de secrets (Docker secrets, Vault, secrets Kubernetes). On ne met **jamais** un secret dans le Dockerfile ni dans l'image.

D.6 Volumes et persistance

Un conteneur est **éphémère** : quand on le supprime, ses données disparaissent. Pour persister (une base de données, par exemple), on utilise un **volume** :

```
docker volume create qb-data # créer un volume nommé
docker run -d --name db \
  -v qb-data:/var/lib/postgresql/data \ # monter le volume dans le conteneur
  -e POSTGRES_PASSWORD=secret postgres:16-alpine

docker volume ls # lister les volumes
docker run --rm -v $(pwd):/app -w /app maven:3.9-eclipse-temurin-21 mvn test
#      ^ bind mount : monter le DOSSIER COURANT de l'hôte dans le conteneur
```

Deux mécanismes à distinguer : le **volume nommé** (géré par Docker, idéal pour les données) et le **bind mount** (`-v /chemin/hôte:/chemin/conteneur` , idéal en développement pour voir ses modifications de code en direct).

D.7 Réseaux entre conteneurs

```
docker network create qb-net # créer un réseau privé
docker run -d --name db --network qb-net postgres:16-alpine
docker run -d --name qb --network qb-net -p 8080:8080 quickbite-api:1.0.0
```

Sur un même réseau Docker, **un conteneur en joint un autre par son nom** : l'API atteint la base via l'hôte `db` (et non `localhost`). C'est ce qui permet à QuickBite d'écrire `jdbc:postgresql://db:5432/...`.

D.8 Docker Compose : tout démarrer ensemble

Plutôt que d'enchaîner les `docker run` , on décrit l'ensemble dans un `docker-compose.yml` . Voici la stack QuickBite (API + base), avec un **healthcheck** :

```
services:
  api:
    build: . # construit depuis le Dockerfile local
    ports: ["8080:8080"]
    environment:
      SPRING_PROFILES_ACTIVE: postgres
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/quickbite
      SPRING_DATASOURCE_USERNAME: app
      SPRING_DATASOURCE_PASSWORD: secret
    depends_on:
      db:
        condition: service_healthy # attend que la base soit VRAIMENT prête
    healthcheck:
      test: ["CMD", "wget", "-q0-", "http://localhost:8080/health"]
      interval: 10s
      timeout: 3s
      retries: 5

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: quickbite
    volumes: ["qb-data:/var/lib/postgresql/data"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app"]
      interval: 5s
      retries: 5

volumes:
  qb-data:
```

```
docker compose up -d --build # construit et démarre tout
docker compose ps            # l'état (et la santé) de chaque service
docker compose logs -f api   # suivre les logs de l'API
docker compose down          # tout arrêter (ajouter -v pour effacer les volumes)
```

Le `depends_on ... condition: service_healthy` règle un piège classique : sans lui, l'API démarre avant que PostgreSQL n'accepte les connexions et plante au lancement. Le **healthcheck** est la bonne façon de gérer l'ordre de démarrage réel.

Exercice — Atelier Docker

Dans le dossier du projet `atelier-quickbite` :

1. Construisez l'image avec `docker build -t quickbite-api:1.0.0` et notez sa **taille** (`docker images`).
2. Lancez-la seule (profil H2 en mémoire) et vérifiez `curl localhost:8080/health`.
3. Lancez la stack complète avec `docker compose up -d --build` ; vérifiez que `db` est `healthy` avant que `api` ne démarre.
4. Ouvrez un shell dans le conteneur API (`docker exec -it <id> sh`) et confirmez qu'il tourne en utilisateur **non-root** (commande `whoami`).
5. Faites `docker compose down`, relancez, et vérifiez que les données ont **survécu** grâce au volume.

Critère de réussite : image < 250 Mo, conteneur non-root, et persistance des données confirmée.

Atelier Kubernetes pratique — déployer pour de vrai

Le Jour 3 a posé les concepts. Ici on **monte un cluster local** et on y déploie QuickBite, on l'expose, on le met à l'échelle, on le met à jour et on le débogue.

Alternatives à connaître

Distributions légères (local/edge) : **K3s**, **k3d**, **kind**, **MicroK8s**. **Orchestrateurs**

concurrents : **Docker Swarm** (bien plus simple, moins puissant) et **HashiCorp Nomad**.

Plateformes d'entreprise : **OpenShift** (Red Hat), **Rancher**. **Kubernetes managé dans le**

cloud : **EKS** (AWS), **GKE** (Google), **AKS** (Azure). Kubernetes est le **standard de fait** : les commandes `kubectl` sont identiques partout.

K.1 Monter un cluster local

Plusieurs options, toutes gratuites et légères :

```
# Option A — minikube (très répandu, démarre une "machine" k8s locale)
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
minikube start --driver=docker
```

```
# Option B — kind (Kubernetes IN Docker, idéal pour la CI)
```

```
# Option C — k3d (k3s léger dans Docker)
```

```
# Installer kubectl (le client en ligne de commande)
```

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install kubectl /usr/local/bin/kubectl
```

```
kubectl version --client
```

```
kubectl get nodes # le(s) nœud(s) de votre cluster
```

K.2 Charger l'image dans le cluster

Sur un cluster local, le cluster ne voit pas automatiquement les images de votre Docker local. On les y charge :

```
minikube image load quickbite-api:1.0.0    # avec minikube
# ou, pour kind : kind load docker-image quickbite-api:1.0.0
```

K.3 Premiers objets : Deployment + Service

Créez `deployment.yaml` :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: quickbite-api
spec:
  replicas: 2                                # 2 copies pour la résilience
  selector:
    matchLabels: { app: quickbite-api }
  template:
    metadata:
      labels: { app: quickbite-api }
    spec:
      containers:
        - name: api
          image: quickbite-api:1.0.0
          imagePullPolicy: IfNotPresent      # utiliser l'image locale chargée
          ports: [{ containerPort: 8080 }]
          env:
            - name: SPRING_PROFILES_ACTIVE
              value: "default"                # H2 en mémoire pour l'atelier
          readinessProbe:                     # PRÊT à recevoir du trafic ?
            httpGet: { path: /actuator/health, port: 8080 }
            initialDelaySeconds: 10
          livenessProbe:                      # toujours VIVANT ? (sinon redémarrage)
            httpGet: { path: /actuator/health, port: 8080 }
            initialDelaySeconds: 20
          resources:
            requests: { cpu: "200m", memory: "256Mi" }
            limits:   { cpu: "1",    memory: "512Mi" }
---
apiVersion: v1
kind: Service
metadata:
  name: quickbite-api
spec:
  selector: { app: quickbite-api }
  ports: [{ port: 80, targetPort: 8080 }]
  type: ClusterIP
```

```
kubectl apply -f deployment.yaml    # créer/mettre à jour les objets
kubectl get pods -w                 # observer les Pods passer à "Running" (-w = watch)
kubectl get deploy,svc,pods         # vue d'ensemble
```

K.4 Accéder à l'application

```
kubectl port-forward svc/quickbite-api 8080:80 # rediriger un port local vers le Service
# dans un autre terminal :
curl http://localhost:8080/health
```

`port-forward` est la façon la plus simple de tester un Service interne depuis votre machine, sans Ingress.

K.5 Configuration et secrets

```
apiVersion: v1
kind: ConfigMap
metadata: { name: quickbite-config }
data:
  APP_PAGE_SIZE: "20"
---
apiVersion: v1
kind: Secret
metadata: { name: quickbite-secret }
type: Opaque
stringData:
  SPRING_DATASOURCE_PASSWORD: "secret-a-changer"
```

On les injecte dans le conteneur via `envFrom` :

```
containers:
- name: api
  image: quickbite-api:1.0.0
  envFrom:
  - configMapRef: { name: quickbite-config }
  - secretRef: { name: quickbite-secret }
```

Rappel du Jour 3 : un `Secret` est **encodé** en base64, **pas chiffré**. Ne le committez jamais en clair dans Git.

K.6 Sondes, mise à jour progressive et rollback

C'est là que Kubernetes brille. Changez l'image vers une nouvelle version et observez le **rolling update** (mise à jour sans coupure, Pod par Pod) :

```
kubectl set image deploy/quickbite-api api=quickbite-api:1.1.0
kubectl rollout status deploy/quickbite-api # suivre la progression du déploiement
kubectl rollout history deploy/quickbite-api # l'historique des versions
kubectl rollout undo deploy/quickbite-api # REVENIR à la version précédente
```

Grâce à la `readinessProbe`, Kubernetes n'envoie du trafic à un nouveau Pod **que** lorsqu'il répond sur `/actuator/health`. Si la nouvelle version est cassée, les anciens Pods restent en service : **pas de coupure**, et un `rollout undo` annule tout en quelques secondes.

K.7 Mise à l'échelle

```
kubectl scale deploy/quickbite-api --replicas=5      # passer à 5 copies manuellement
kubectl get pods                                     # voir les nouveaux Pods apparaître

# Mise à l'échelle AUTOMATIQUE selon le CPU (HPA) :
kubectl autoscale deploy/quickbite-api --min=2 --max=10 --cpu-percent=70
kubectl get hpa
```

L'**HPA** (*Horizontal Pod Autoscaler*) ajoute ou retire des Pods automatiquement quand la charge CPU dépasse/descend sous le seuil — c'est l'élasticité dont on rêvait au Jour 3.

K.8 Débuguer dans Kubernetes

```
kubectl describe pod <nom>      # ★ l'OUTIL N°1 : voir les "Events" en bas
kubectl logs <pod>               # les logs applicatifs
kubectl logs -f <pod>           # les suivre en direct
kubectl logs <pod> --previous    # les logs du conteneur AVANT son dernier crash
kubectl exec -it <pod> -- sh      # un shell dans le Pod
kubectl get events --sort-by=.lastTimestamp # tous les événements récents du cluster
```

⚠ Décrypter les statuts d'un Pod

Un Pod qui ne démarre pas affiche un statut parlant. **ImagePullBackOff** : l'image est introuvable (mauvais nom, ou pas chargée dans le cluster — voir K.2). **CrashLoopBackOff** : le conteneur démarre puis plante en boucle (lisez `kubectl logs --previous`). **Pending** : aucun nœud ne peut l'accueillir (ressources insuffisantes). Le réflexe est **toujours** le même : `kubectl describe pod <nom>`, puis lire la section « Events ».

K.9 Exposer via un Ingress

```
minikube addons enable ingress      # activer le contrôleur Ingress
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: quickbite-ingress
spec:
  rules:
    - host: quickbite.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: quickbite-api
                port: { number: 80 }
```



```
kubectl apply -f ingress.yaml
echo "$(minikube ip) quickbite.local" | sudo tee -a /etc/hosts # résolution du nom
curl http://quickbite.local/health
```

Exercice — Atelier Kubernetes

1. Démarrez minikube et chargez votre image `quickbite-api:1.0.0`.
2. Appliquez le Deployment + Service ; vérifiez que **2 Pods** sont `Running` et que les sondes passent.
3. Accédez à `/health` via `kubectl port-forward`.
4. Provoquez une panne : `kubectl delete pod <nom>` et observez Kubernetes en **recréer un** automatiquement (auto-réparation).
5. Faites un `set image` vers une image inexistante, constatez le **ImagePullBackOff**, diagnostiquez avec `describe`, puis `rollout undo`.
6. Passez à 5 réplicas, puis exposez l'app via Ingress.

Réflexe à ancrer : en cas de souci, votre premier geste est toujours `kubectl describe pod` + lecture des Events.

Atelier GitHub — la plateforme de collaboration

Git gère les versions ; **GitHub** est la plateforme qui transforme Git en outil d'équipe : hébergement, revues de code, suivi des tâches, automatisation. C'est aussi là que vivra notre CI/CD (atelier suivant).

Alternatives à connaître

GitLab : CI/CD intégrée très complète, auto-hébergeable. **Bitbucket** : intégré à l'écosystème Atlassian (donc à **Jira**). **Azure DevOps** : suite Microsoft (Repos + Pipelines + Boards). **Gitea** / **Forgejo** : légers et auto-hébergés. Tous reposent sur **Git** : les commandes `git` sont les mêmes ; seules changent l'interface et les fonctions d'équipe.

G.1 Créer et alimenter un dépôt

```
gh auth login # s'authentifier avec le GitHub CLI
gh repo create quickbite --public --source=. --remote=origin --push
# ou à la main :
git init
git add .
git commit -m "feat: initial commit du projet QuickBite"
git branch -M main
git remote add origin https://github.com/<vous>/quickbite.git
git push -u origin main
```

Trois fichiers à ne jamais oublier à la racine : un `README.md` (présentation, comment lancer le projet), un `.gitignore` (ce qu'on ne versionne **pas** : `target/`, `.env`, `*.class`...), et une `LICENSE`.

⚠ **target/ et secrets : hors de Git**

Pour QuickBite (Maven), **target/** contient des fichiers compilés **régénérables** — il ne doit **jamais** être versionné. Idem pour tout fichier de secrets (**.env** , clés). Si un secret a déjà été poussé, le retirer du dernier commit **ne suffit pas** : il reste dans l'historique. Il faut le **révoquer** et le considérer comme compromis.

G.2 Le flux de travail par branches et Pull Requests

Le cœur de la collaboration sur GitHub (le **GitHub Flow**, vu au Jour 1) :

```
git switch -c feature/paiement      # 1. créer une branche dédiée à la fonctionnalité
# ... on code, on commit ...
git push -u origin feature/paiement # 2. publier la branche
gh pr create --fill                 # 3. ouvrir une Pull Request
# 4. un collègue relit, commente, approuve
gh pr merge --squash --delete-branch # 5. fusionner puis supprimer la branche
```

Une **Pull Request (PR)** n'est pas qu'un bouton « fusionner » : c'est l'**espace de revue**. On y discute le code ligne par ligne, on y voit le résultat des tests automatiques (la CI), et on n'autorise la fusion que lorsque tout est vert et approuvé.

G.3 Issues, labels, milestones, Projects

GitHub n'est pas qu'un dépôt de code, c'est aussi un outil de **gestion de projet** — qui rejoint directement le Module 1 :

- Les **Issues** : bugs, tâches, User Stories. On les **assigne**, on les **étiquette** (**bug** , **enhancement** , **priority:high**).
- Les **Milestones** : regrouper des issues vers un objectif (par ex. un sprint, une version).
- Les **Projects** : des **tableaux Kanban** (les colonnes « À faire / En cours / Terminé » du Kanban (Module 1)) directement reliés aux issues et PR.
- Un commit ou une PR qui mentionne **Closes #42** **ferme automatiquement** l'issue 42 à la fusion.

G.4 Protéger la branche **main**

C'est le réglage qui rend la CI **vraiment** utile. Dans **Settings → Branches → Add rule** sur **main** :

- ☒ **Require a pull request before merging** — interdire les push directs sur **main**.
- ☒ **Require approvals** (au moins 1 relecteur).
- ☒ **Require status checks to pass** — **interdire la fusion si la CI est rouge**.
- ☒ **Require branches to be up to date** — la branche doit être à jour avant fusion.

Le fichier **CODEOWNERS** désigne des relecteurs obligatoires par zone du code :

```
# .github/CODEOWNERS
*.java      @equipe-backend
/.github/    @equipe-devops
/k8s/        @equipe-devops
```

G.5 Releases, tags et secrets du dépôt

```
git tag -a v1.0.0 -m "Première version stable"
git push origin v1.0.0
gh release create v1.0.0 --generate-notes # génère les notes de version depuis les PR
```

Deux réglages dans **Settings** qui serviront à la CI/CD :

- **Secrets and variables → Actions** : on y stocke les valeurs sensibles (tokens, mots de passe) que les workflows liront **sans jamais les exposer** dans le code.
- **Environments** (**staging** , **production**) : on peut exiger une **approbation manuelle** avant qu'un déploiement vers **production** ne s'exécute.

Exercice — Atelier GitHub

1. Poussiez QuickBite sur un dépôt GitHub avec un **README** et un **.gitignore** Java correct (**target/** exclu).
2. Activez la **protection de branche** sur **main** : PR obligatoire + 1 approbation + status checks requis.
3. Créez une **Issue** « Ajouter un endpoint /menu », ouvrez une branche, faites une **PR** qui la référence avec **Closes #1**.
4. Constatez que vous **ne pouvez pas** pousser directement sur **main**, puis fusionnez via la PR.
5. Créez un **tag** **v1.0.0** et une **release**.

Objectif : un dépôt où **main** est protégée et où tout changement passe par une PR relue.

Atelier GitHub Actions — CI/CD de bout en bout

On automatise enfin tout : à chaque push, **tester** ; sur la branche **main**, **construire et publier** l'image ; sur un tag, **déployer**. C'est l'aboutissement concret du Module 2.

Alternatives à connaître

GitLab CI (**.gitlab-ci.yml**), **Jenkins** (le vétéran auto-hébergé, ultra-flexible, pipelines en Groovy), **CircleCI**, **Travis CI**, **Azure Pipelines**, **Drone**, **Tekton** (CI native Kubernetes). Le **principe est universel** : des étapes déclenchées par un événement, décrites en YAML (sauf Jenkins, historiquement scripté). Ce que vous apprenez ici se transpose presque tel quel ailleurs.

A.1 L'anatomie d'un workflow

Un workflow est un fichier YAML dans `.github/workflows/`. Le vocabulaire :

Terme	Définition
Workflow	le fichier complet, déclenché par un ou plusieurs événements
Event (<code>on</code>)	ce qui le déclenche : <code>push</code> , <code>pull_request</code> , <code>tag</code> , manuel, planifié
Job	un groupe d'étapes qui s'exécute sur une machine ; les jobs tournent en parallèle par défaut
Runner	la machine qui exécute le job (<code>ubuntu-latest</code> fourni par GitHub)
Step	une étape : soit une commande (<code>run</code>), soit une action réutilisable (<code>uses</code>)
Action	un composant réutilisable publié par la communauté (ex. <code>actions/checkout</code>)

A.2 Le pipeline CI : tester QuickBite à chaque push

Créez `.github/workflows/ci.yml`. Comme QuickBite est en **Maven/Java 21**, on s'appuie sur l'action `setup-java` avec cache Maven intégré :

```
name: CI
on:
  push:
    branches: [ main ]
  pull_request:                # se déclenche aussi sur chaque PR

jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4          # 1) récupérer le code
      - uses: actions/setup-java@v4        # 2) installer le JDK 21
        with:
          distribution: temurin
          java-version: '21'
          cache: maven                    # met en cache ~/.m2 (build plus rapide)
      - run: mvn -B verify                # 3) compiler + lancer TOUS les tests
      - uses: actions/upload-artifact@v4   # 4) conserver le rapport de tests
        if: always()                      # même si l'étape précédente a échoué
        with:
          name: test-reports
          path: target/surefire-reports/
```

`mvn -B verify` compile, lance les tests unitaires **et** d'intégration, et échoue (code $\neq$ 0) au premier test rouge — ce qui marque le job en **rouge**. Couplé à la protection de branche (atelier précédent), cela **bloque** la fusion d'un code cassé.

A.3 Matrice : tester sur plusieurs versions

```
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        java: [ '17', '21' ]      # exécute le job une fois par version
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '${{ matrix.java }}', cache: maven }
      - run: mvn -B verify
```

La **matrice** lance le même job en parallèle pour chaque combinaison — utile pour garantir la compatibilité sur plusieurs versions de langage ou d'OS.

A.4 Construire et publier l'image dans GHCR

Sur la branche `main`, on construit l'image Docker et on la **pousse dans GitHub Container Registry** (`ghcr.io`), gratuit et intégré. Le `GITHUB_TOKEN` est fourni automatiquement :

```
build-push:
  needs: build-test          # ne démarre QUE si les tests passent
  if: github.ref == 'refs/heads/main' # uniquement sur main
  runs-on: ubuntu-latest
  permissions:
    contents: read
    packages: write          # autorise l'écriture dans GHCR
  steps:
    - uses: actions/checkout@v4
    - uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: ${GITHUB_ACTOR}
        password: ${GITHUB_TOKEN}
    - uses: docker/metadata-action@v5 # génère des tags propres (sha, latest...)
      id: meta
      with:
        images: ghcr.io/${GITHUB_REPOSITORY}
    - uses: docker/build-push-action@v6
      with:
        context: .
        push: true
        tags: ${GITHUB_REF_NAME}
        cache-from: type=gha      # réutiliser le cache de build entre exécutions
        cache-to: type=gha,mode=max
```

`needs: build-test` exprime la **dépendance** : on ne publie une image que si elle a passé les tests. Le `if:` restreint ce job à `main` (on ne publie pas depuis une PR).

A.5 Le job de déploiement, avec approbation

```

deploy:
  needs: build-push
  runs-on: ubuntu-latest
  environment: production # ★ déclenche l'approbation manuelle configurée sur cet env
  steps:
    - uses: actions/checkout@v4
    - name: Déployer sur Kubernetes
      run: |
        echo "${{ secrets.KUBECONFIG }}" > kubeconfig
        export KUBECONFIG=$PWD/kubeconfig
        kubectl set image deploy/quickbite-api \
          api=ghcr.io/${{ github.repository }}:${{ github.sha }} -n prod
        kubectl rollout status deploy/quickbite-api -n prod

```

En liant le job à l'`environment: production` (configuré en G.5 pour exiger une approbation), GitHub **met le déploiement en pause** jusqu'à ce qu'un humain clique sur « Approve ». C'est la frontière nette entre **livraison continue** (déploiement validé manuellement) et **déploiement continu** (entièrement automatique) du Jour 1.

A.6 Secrets, variables et bonnes pratiques

- **Secrets** : `${{ secrets.MON_SECRET }}`, définis dans `Settings → Secrets`. Masqués dans les logs, jamais en clair dans le YAML.
- **Permissions minimales** : commencez par `permissions: { contents: read }` au niveau du workflow, et n'élargissez (`packages: write` ...) que par job, là où c'est nécessaire — principe du moindre privilège.
- **Épinglez vos actions** : `actions/checkout@v4` (ou mieux, un SHA) plutôt que `@main`, pour la reproductibilité et la sécurité.
- **concurrency** : annule les exécutions en double sur une même branche.

```

permissions:
  contents: read
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true # annule l'ancien run si on repousse

```

A.7 Déclencheurs au-delà du push

```

on:
  push:
    tags: [ 'v*' ] # sur un tag de version (release)
  workflow_dispatch: # bouton "Run workflow" manuel dans l'UI
  schedule:
    - cron: '0 3 * * 1' # tous les lundis à 3h (ex. scan de sécurité)

```

💡 La stratégie CI/CD complète de QuickBite

On combine tout en une chaîne lisible : **sur chaque PR** → tests (rien d'autre, on ne publie pas) ; **sur push vers main** → tests puis build & push de l'image taguée par le SHA du commit ; **sur un tag v*** → déploiement en production après approbation manuelle. Chaque étage ne s'exécute que si le précédent est vert (`needs`), et la branche `main` protégée garantit que rien ne fusionne sans CI verte. C'est exactement la définition de DORA : petits changements, fréquents, sûrs.

🔧 Exercice — Atelier final : du commit à la production

Sur votre dépôt QuickBite, assemblez le pipeline complet :

1. **CI** : workflow `ci.yml` qui lance `mvn -B verify` sur chaque PR et push. Cassez un test, ouvrez une PR, et constatez la fusion **bloquée**.
2. **Build & push** : sur `main`, construisez l'image et publiez-la dans `ghcr.io`. Vérifiez qu'elle apparaît dans l'onglet « Packages ».
3. **Cache** : confirmez que le second run est nettement plus rapide grâce au cache Maven et au cache de build Docker.
4. **Environnement protégé** : créez un environnement `production` avec approbation requise, et un job `deploy` qui s'y rattache.
5. **Release** : poussez un tag `v1.1.0` et observez la chaîne se dérouler jusqu'à l'attente d'approbation.

Démonstration finale : racontez le trajet complet d'une modification de code — du `git commit` jusqu'à l'image déployée — en nommant, à chaque étape, l'outil et le garde-fou (test, revue, branche protégée, approbation) qui sécurise le passage. C'est la synthèse de tout le Module 2.

Atelier DevSecOps — sécuriser la chaîne de livraison

On sait livrer vite (ateliers précédents) et l'on apprendra à sécuriser le code applicatif (Module 3). Le **DevSecOps** relie les deux : intégrer la sécurité **dans le pipeline**, automatiquement et **au plus tôt** — c'est le principe du « **shift left** » (déplacer la sécurité vers la gauche, tôt dans le cycle, au lieu de la subir en production).

🔧 Alternatives à connaître

En complément des outils du tableau ci-dessous : **Snyk** et **GitHub Advanced Security** couvrent dépendances + code + conteneurs en une offre ; **Grype** est une alternative à Trivy pour le scan d'image ; **tfsec** / **Checkov** scannent le code Terraform ; **OWASP ZAP** fait du **DAST** (test dynamique, sur l'application en marche). L'important n'est pas l'outil exact, mais le **principe** : automatiser chaque contrôle dans le pipeline.

S.1 Les couches à scanner

Cible	Risque	Outil typique
Dépendances	une bibliothèque tierce vulnérable (CVE)	Dependabot, Trivy, OWASP Dependency-Check
Code	injection, secret en dur, mauvaise pratique	CodeQL, SonarQube (SAST)
Secrets	clé/mot de passe committé par erreur	GitHub secret scanning, gitleaks
Image	OS et libs de l'image porteurs de failles	Trivy, Gripe
Infra	configuration dangereuse (port ouvert...)	tfsec, Checkov

S.2 Scanner les dépendances

La majorité des vulnérabilités proviennent des **bibliothèques tierces**, pas de votre code. **Dependabot** (intégré et gratuit sur GitHub) ouvre automatiquement des Pull Requests quand une dépendance vulnérable reçoit un correctif. On l'active avec `.github/dependabot.yml` :

```
version: 2
updates:
  - package-ecosystem: maven          # les dépendances Java de QuickBite
    directory: "/"
    schedule: { interval: weekly }
  - package-ecosystem: docker         # l'image de base du Dockerfile
    directory: "/"
    schedule: { interval: weekly }
  - package-ecosystem: github-actions # les actions épinglées dans les workflows
    directory: "/"
    schedule: { interval: weekly }
```

S.3 Analyser le code (SAST)

Le **SAST** (*Static Application Security Testing*) lit le code source **sans l'exécuter** pour y repérer des failles (injection SQL, secret en dur, chemin non validé...). **CodeQL** (GitHub) est gratuit sur les dépôts publics :

```
- uses: github/codeql-action/init@v3
  with: { languages: java }
- run: mvn -B compile
- uses: github/codeql-action/analyze@v3
```

SonarQube complète l'analyse par la **qualité** (bugs, *code smells*, dette technique) et agrège la couverture de tests vue à l'atelier précédent.

S.4 Scanner l'image Docker (Trivy)

Une image embarque un système et des bibliothèques qui ont **leurs propres** vulnérabilités, indépendantes de votre code. **Trivy** les détecte :

```
trivy image ghcr.io/monuser/quickbite-api:1.0.0
```

Intégré au pipeline, juste après le build de l'image :

```
- uses: aquasecurity/trivy-action@master
  with:
    image-ref: ghcr.io/${{ github.repository }}:${{ github.sha }}
    severity: CRITICAL,HIGH
    exit-code: '1'          # FAIT ÉCHOUER le pipeline si une faille critique est trouvée
```

S.5 Empêcher les secrets de fuir

Un mot de passe committé par erreur est **compromis pour toujours** (il reste dans l'historique Git). **GitHub secret scanning** et **gitleaks** détectent les secrets dans le code et dans l'historique, avant qu'ils ne fuient.

! Un secret poussé est un secret brûlé

Si une clé d'API ou un mot de passe atterrit dans un commit, le retirer dans un commit suivant **ne suffit pas** : il reste consultable dans l'historique et a pu être aspiré en quelques secondes par des robots qui scrutent GitHub en continu. La seule réaction correcte est de **révoquer immédiatement** le secret et d'en générer un nouveau. La prévention (secret scanning + `.gitignore` + variables d'environnement) vaut infiniment mieux que la guérison.

S.6 Tracer ce qu'on livre : SBOM et signature

- Un **SBOM** (*Software Bill of Materials*) est la **liste exhaustive** des composants d'une image, générée par un outil comme **Syft**. Quand une faille majeure éclate (souvenez-vous de Log4Shell), un SBOM permet de savoir **en quelques minutes** si l'on est concerné, au lieu de fouiller chaque projet à la main.
- La **signature** d'image (avec **cosign**) prouve cryptographiquement qu'une image provient bien de **votre** pipeline et n'a pas été altérée entre la construction et le déploiement.

💡 Où placer chaque contrôle dans le pipeline

Une stratégie efficace : sur **chaque Pull Request**, lancez les contrôles **rapides** (scan de dépendances, SAST, secrets) ; **après le build de l'image**, lancez le scan Trivy ; **au moment du push** dans le registre, générez le SBOM et signez l'image. Règle d'or : **un scan qui ne bloque jamais ne sert à rien**. Configurez l'échec du pipeline sur les sévérités hautes — sinon les alertes deviennent du bruit qu'on finit par ignorer.

Exercice — Atelier DevSecOps

Sur le dépôt QuickBite :

1. Activez **Dependabot** avec le fichier ci-dessus et observez les PR de mise à jour qu'il propose.
2. Ajoutez un job **Trivy** qui scanne l'image construite et **fait échouer** le pipeline sur une faille **CRITICAL**.
3. Activez le **secret scanning** du dépôt, puis tentez (sur une branche jetable) de committer une fausse clé pour voir l'alerte se déclencher.
4. Ajoutez l'analyse **CodeQL** au workflow et lisez les éventuelles alertes dans l'onglet « Security ».

Objectif : que la sécurité devienne une étape **automatique** du pipeline, et non une revue manuelle de fin de projet.

Atelier Terraform — l'infrastructure en tant que code (IaC)

Jusqu'ici, on a déployé **sur** une infrastructure existante (un cluster, un registre). Mais cette infrastructure elle-même — serveurs, réseaux, bases managées, clusters — comment la crée-t-on ? Si c'est **à la main, en cliquant dans une console web** (le « *ClickOps* »), le résultat est non reproductible, non versionné, et impossible à recréer à l'identique après un incident. L'**Infrastructure as Code** (IaC) décrit l'infrastructure dans des **fichiers versionnés**.

Alternatives à connaître

OpenTofu : fork 100 % open-source de Terraform (même langage HCL), né après le changement de licence — interchangeable. **Pulumi** : l'IaC écrite dans un **vrai langage** (Python, TypeScript, Go). **Ansible** : surtout de la *configuration* de serveurs, mais fait aussi du provisioning (sans **state**). **CloudFormation** (AWS) et **Bicep** (Azure) : spécifiques à un cloud. Terraform / OpenTofu restent les plus **multi-cloud**.

I.1 Déclaratif et idempotent

Comme Kubernetes, **Terraform** est **déclaratif** : on décrit l'**état souhaité** de l'infrastructure, et Terraform calcule les actions nécessaires pour l'atteindre. Relancer la même opération quand rien n'a changé ne fait **rien** (idempotence) — on retrouve exactement la logique du Jour 3.

I.2 Les concepts

Terme	Rôle
Provider	un plugin vers une plateforme : AWS, Azure, GCP, Docker, Kubernetes...
Resource	un objet à gérer : une VM, un réseau, un conteneur, un bucket
State	le fichier où Terraform mémorise ce qu'il a déjà créé
Plan	l'aperçu des changements à venir, sans rien modifier
Apply	l'exécution effective des changements

I.3 Un exemple concret, en local (provider Docker)

Pour pratiquer **sans compte cloud**, le provider **Docker** suffit. Créez `main.tf` :

```
terraform {
  required_providers {
    docker = { source = "kreuzwerker/docker", version = "~> 3.0" }
  }
}
provider "docker" {}

resource "docker_image" "quickbite" {
  name = "quickbite-api:1.0.0" # l'image construite à l'atelier Docker
}

resource "docker_container" "quickbite" {
  name = "quickbite"
  image = docker_image.quickbite.image_id
  ports {
    internal = 8080
    external = 8080
  }
}
```

Le cycle de travail est toujours le même :

```
terraform init    # télécharge le provider
terraform plan    # montre CE QUI VA changer, sans rien faire (le "dry run")
terraform apply   # crée/modifie l'infrastructure, après confirmation
terraform destroy # détruit proprement tout ce que Terraform a créé
```



plan avant apply, toujours

`terraform plan` est votre garde-fou : il liste, ressource par ressource, ce qui sera **créé (+)**, **modifié (~)** ou **détruit (-)**. Lisez-le *vraiment* avant d'appliquer : un `~` anodin sur une base de données peut en réalité signifier « je la détruis et j'en recrée une vide ». Le `plan` est ce qui transforme une opération risquée en décision éclairée.

1.4 Variables et sorties

On paramètre l'infrastructure (par environnement) avec des **variables**, et on récupère des valeurs utiles avec des **outputs** :

```
variable "app_version" {
  description = "Version de l'image à déployer"
  default     = "1.0.0"
}

output "url" {
  value = "http://localhost:8080/health"
}
```

1.5 Les modules

Un **module** est un paquet réutilisable de ressources — l'équivalent d'une fonction pour l'infrastructure. On factorise ainsi une infra type (réseau + base + application) et on la réinstancie pour **dev**, **staging** et **prod** en ne changeant que quelques variables, exactement comme un *chart* Helm paramètre des manifestes Kubernetes.

1.6 Les bonnes pratiques

! Le fichier **state** est sensible — ne le committez jamais

terraform.tfstate contient l'image complète de votre infrastructure **et, parfois, des secrets en clair** (mots de passe de bases générés...). Trois règles : (1) ajoutez-le au **.gitignore** et stockez-le **à distance** (S3, Terraform Cloud) avec verrouillage pour éviter que deux personnes l'écrasent en même temps ; (2) lancez **terraform plan** **en CI sur chaque PR** et **apply** seulement après revue — un GitOps de l'infrastructure ; (3) ne modifiez **jamais à la main** une ressource gérée par Terraform, sinon le réel « dérive » du **state** et le prochain **apply** réserve de mauvaises surprises.

Exercice — Atelier Terraform

1. Installez Terraform et écrivez le **main.tf** ci-dessus (provider Docker).
2. Enchaînez **init** → **plan** → **apply** et vérifiez que QuickBite répond sur **/health**.
3. **Lisez attentivement** la sortie de **plan** : identifiez les **+** (création).
4. Passez le port externe en variable, faites un **plan** et observez le **~** (modification).
5. Ajoutez **terraform.tfstate** au **.gitignore**, puis détruisez l'infra avec **destroy**.

Objectif : comprendre qu'une infrastructure se décrit, se versionne et se recrée à l'identique — comme du code.

Récapitulatif de la partie pratique

Vous avez maintenant manipulé, de vos mains, toute la chaîne : **Linux** est le terrain (terminal, fichiers, processus, services, scripts) ; vous avez **conçu et documenté** l'API (REST, OpenAPI/Swagger) puis **verrouillé sa qualité par des tests** automatisés (JUnit,

Mockito, Testcontainers) et **versionné le schéma de base** avec Flyway ; **Docker** empaquète QuickBite dans une image légère et reproductible ; **Kubernetes** l'exécute de façon résiliente, scalable et auto-réparatrice ; **GitHub** structure la collaboration (branches, PR, revues, branche protégée) ; **GitHub Actions** automatise tout, du test à la mise en production sous contrôle ; le **DevSecOps** y injecte la sécurité au plus tôt (dépendances, image, secrets) ; et **Terraform** provisionne l'infrastructure elle-même de façon reproductible. Aucun de ces outils n'a de valeur isolément : leur force vient de la **chaîne** qu'ils forment, au service du même objectif que le Module 1 — livrer de la valeur, vite et sûrement.

Évaluation finale du Module 2

Partie A — QCM

1. CAMS signifie : **Culture, Automation, Measurement, Sharing.**
2. CI vs CD : ?
3. Multi-stage Docker sert à : **réduire la taille de l'image finale.**
4. Un Deployment gère : **les ReplicaSet / les mises à jour et le rollback.**
5. Helm sert à : **packager/paramétrer des manifestes Kubernetes.**
6. GitOps : la source de vérité est : **Git.**
7. Prometheus collecte des : **métriques.**
8. Les 4 signaux dorés : **latence, trafic, erreurs, saturation.**

Partie B — Étude de cas (atelier noté)

On évalue l'atelier final sur la grille suivante :

Critère	Points
Pipeline CI fonctionnel (tests bloquants)	4
Image conteneurisée propre (multi-stage, sécurité)	4
Déploiement Kubernetes via Helm	5
Exposition + configuration (Ingress, ConfigMap/Secret)	3
Observabilité (dashboard)	2
Bonus GitOps (Argo CD)	+2

✓ Clôture du Module 2

Reliez explicitement au Module 1 : la chaîne DevOps n'a de valeur que si elle sert un **flux de valeur** agile (livrer tôt et souvent). Annoncez le Module 3 : sécuriser l'application que l'on vient d'apprendre à livrer.